

Ricorsione e Liste Linkate

Liste Strutture Ricorsive, Operazioni Ricorsive su Liste
Linkate

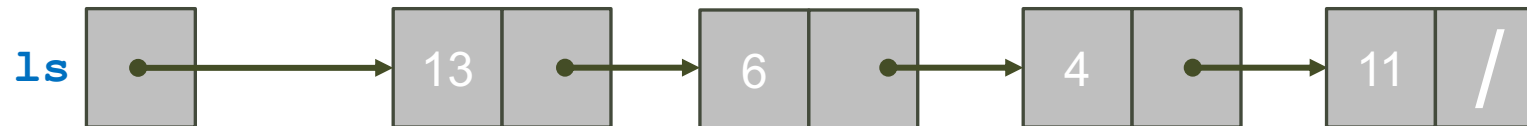
Questi lucidi sono basati sui lucidi di Programmazione 2 del Prof. Damiani e della Prof.ssa Baroglio



Ricorsione e Liste Linkate

Liste Linkate: Ricorsive per Natura

- Le liste linkate si prestano a essere manipolate tramite funzioni ricorsive:
 - Per via della propria natura ricorsiva
 - Una lista:
 - è vuota
 - oppure è un elemento seguito da una lista



Liste Linkate: Struttura Base

- Funzioni ricorsive dobbiamo individuare:

-
-
-
-

```
int funzRicorsiva(List L) {
```

```
}
```



Liste Linkate: Struttura Base

- Funzioni ricorsive dobbiamo individuare:

- Caso Base

-

-

-

```
int funzRicorsiva(List L) {
```

```
}
```



Liste Linkate: Struttura Base

- Funzioni ricorsive dobbiamo individuare:
 - Caso Base
 - Lista vuota

```
int funzRicorsiva(List L) {
```



Liste Linkate: Struttura Base

- Funzioni ricorsive dobbiamo individuare:

- Caso Base
 - Lista vuota
 -
 -

```
int funzRicorsiva(List L) {  
    if (L == NULL) {  
        // CASO BASE  
    }  
}
```



Liste Linkate: Struttura Base

- Funzioni ricorsive dobbiamo individuare:
 - Caso Base
 - Lista vuota
 - Caso induttivo con chiamata ricorsiva
 -

```
int funzRicorsiva(List L) {  
    if (L == NULL) {  
        // CASO BASE  
    }  
}
```



Liste Linkate: Struttura Base

- Funzioni ricorsive dobbiamo individuare:
 - Caso Base
 - Lista vuota
 - Caso induttivo con chiamata ricorsiva
 - Lista rimanente

```
int funzRicorsiva(List L) {  
    if (L == NULL) {  
        // CASO BASE  
    }  
}
```



Liste Linkate: Struttura Base

- Funzioni ricorsive dobbiamo individuare:
 - Caso Base
 - Lista vuota
 - Caso induttivo con chiamata ricorsiva
 - Lista rimanente

```
int funzRicorsiva(List L) {
    if (L == NULL) {
        // CASO BASE
    }

    // CASO INDUTTIVO
    /* INSERIRE QUI IL CODICE CHE
    VISITA IL NODO ls */
    return funzRicorsiva(L->next);
}
```

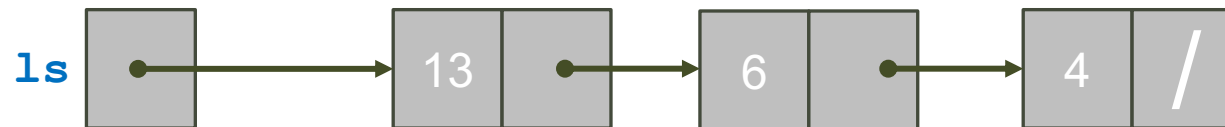


Esempio: printList()

- Stampa elementi lista

```
void printList(List L) {
```

```
}
```



Esempio: printList()

- Stampa elementi lista

- Caso base?

-

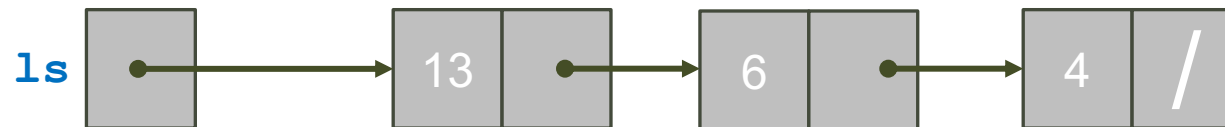
-

-

-

```
void printList(List L) {
```

```
}
```

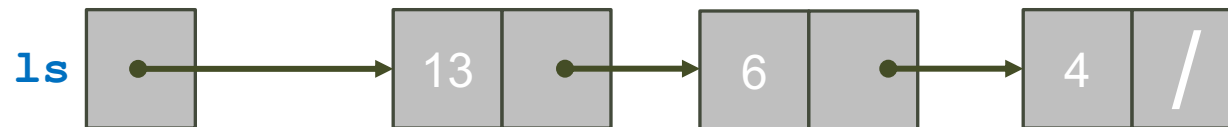


Esempio: printList()



- Stampa elementi lista
 - Caso base?
 - Lista vuota
 -
 -
 -

```
void printList(List L) {  
  
  
  
  
  
}
```



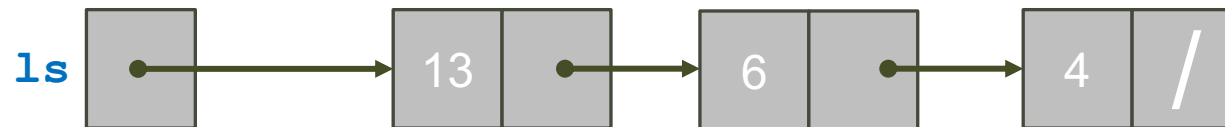
Esempio: printList()



- Stampa elementi lista

- Caso base?
 - Lista vuota
 - Ho finito
-
-

```
void printList(List L) {  
    // Caso base  
    if (L == NULL) return;  
  
}
```



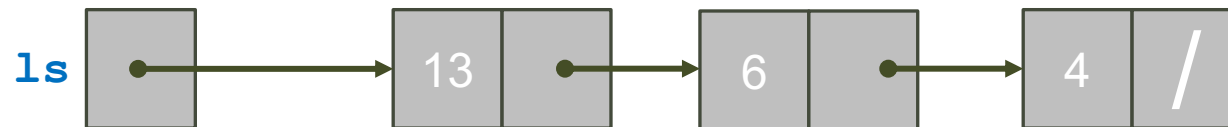
Esempio: printList()



- Stampa elementi lista

- Caso base?
 - Lista vuota
 - Ho finito
- Caso induttivo?
 -

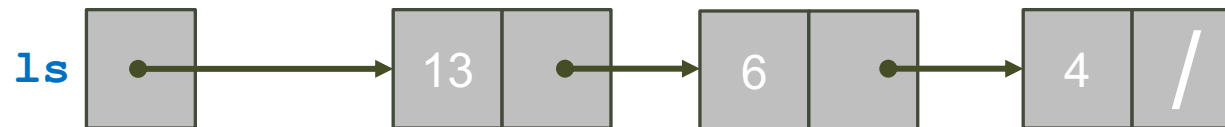
```
void printList(List L) {  
    // Caso base  
    if (L == NULL) return;  
  
}
```



Esempio: printList()

- Stampa elementi lista
 - Caso base?
 - Lista vuota
 - Ho finito
 - Caso induttivo?
 - Stampo e passo all'elemento successivo

```
void printList(List L) {  
    // Caso base  
    if (L == NULL) return;  
  
    //Caso induttivo  
    printf("%d ", L->data);  
    return printList(L->next);  
}
```

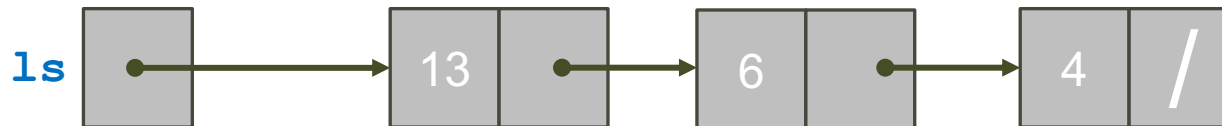


Esempio: printList()

- Stampa elementi lista
 - Caso base?
 - Lista vuota
 - Ho finito
 - Caso induttivo?
 - Stampo e passo all'elemento successivo

```
void printList(List L) {  
    // Caso base  
    if (L == NULL) return;  
  
    //Caso induttivo  
    printf("%d ", L->data);  
    return printList(L->next);  
}
```

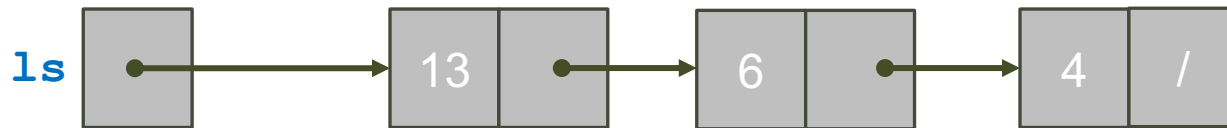
Progresso



Simulazione: printList()



```
void printList(List L) {  
    // Caso base  
    if (L == NULL) return;  
  
    //Caso induttivo  
    printf("%d ", L->data);  
    return printList(L->next);  
}
```

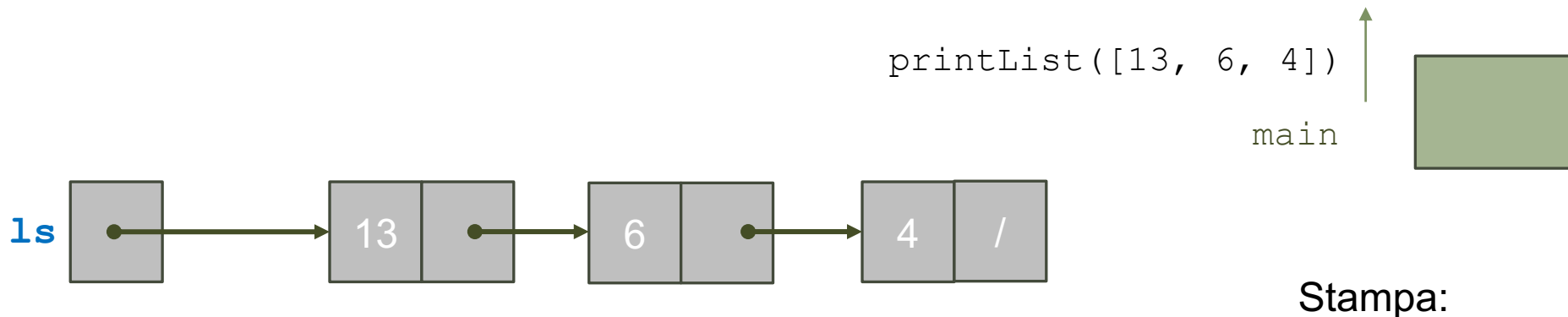


Stampa:

Simulazione: printList()



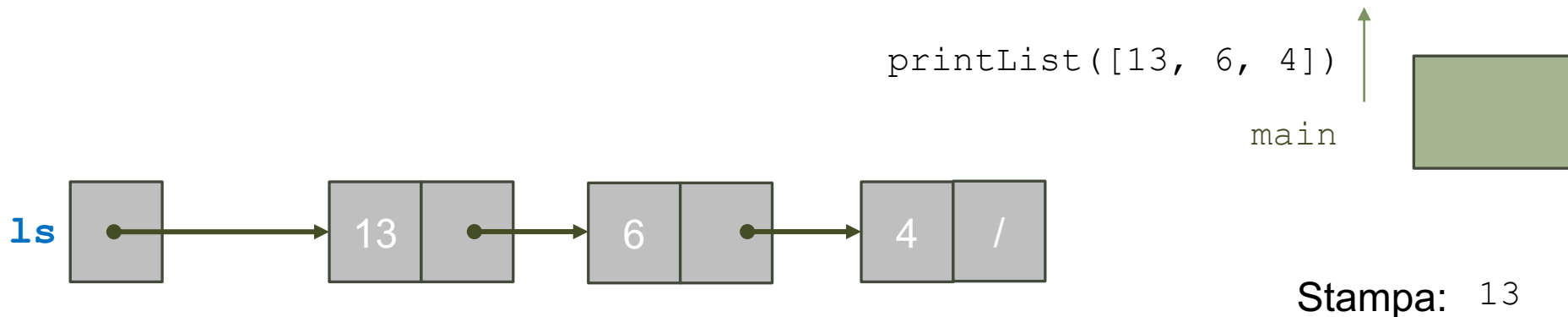
```
void printList(List L) {  
    // Caso base  
    if (L == NULL) return;  
  
    //Caso induttivo  
    printf("%d ", L->data);  
    return printList(L->next);  
}
```



Simulazione: printList()



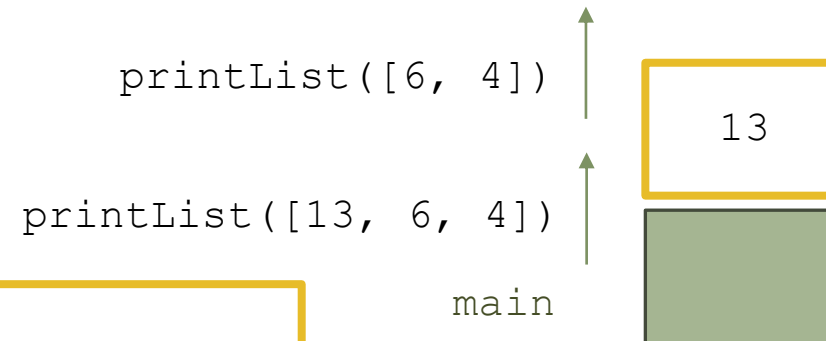
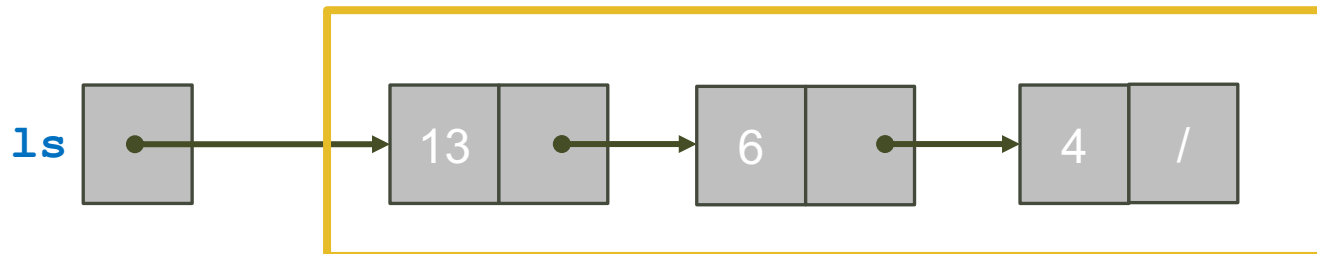
```
void printList(List L) {  
    // Caso base  
    if (L == NULL) return;  
  
    //Caso induttivo  
    printf("%d ", L->data);  
    return printList(L->next);  
}
```



Simulazione: printList()



```
void printList(List L) {  
    // Caso base  
    if (L == NULL) return;  
  
    //Caso induttivo  
    printf("%d ", L->data);  
    return printList(L->next);  
}
```

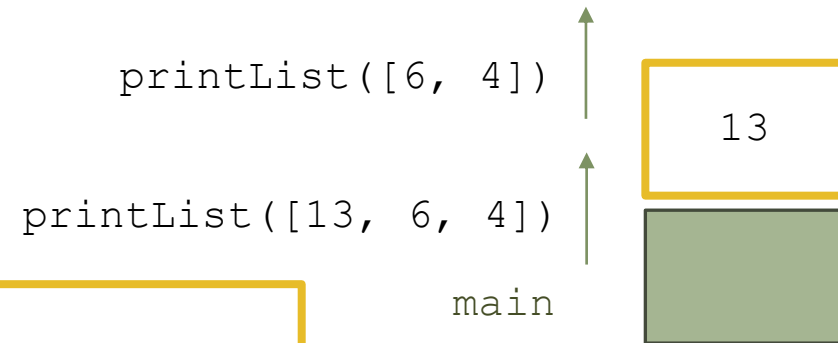
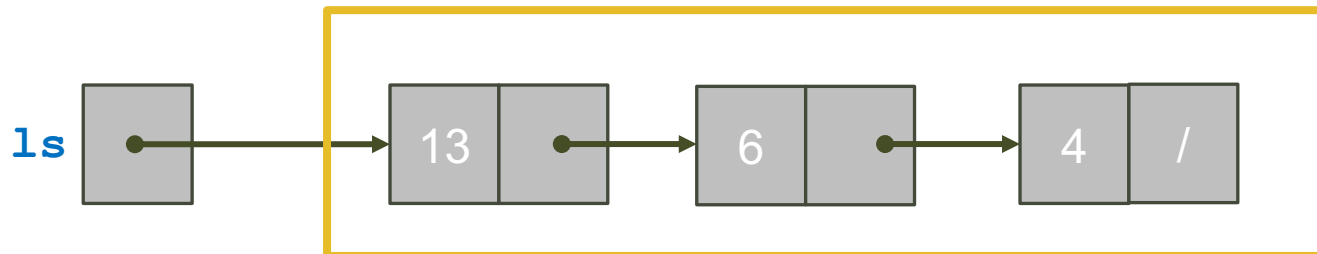


Stampa: 13

Simulazione: printList()



```
void printList(List L) {  
    // Caso base  
    if (L == NULL) return;  
  
    //Caso induttivo  
    printf("%d ", L->data);  
    return printList(L->next);  
}
```

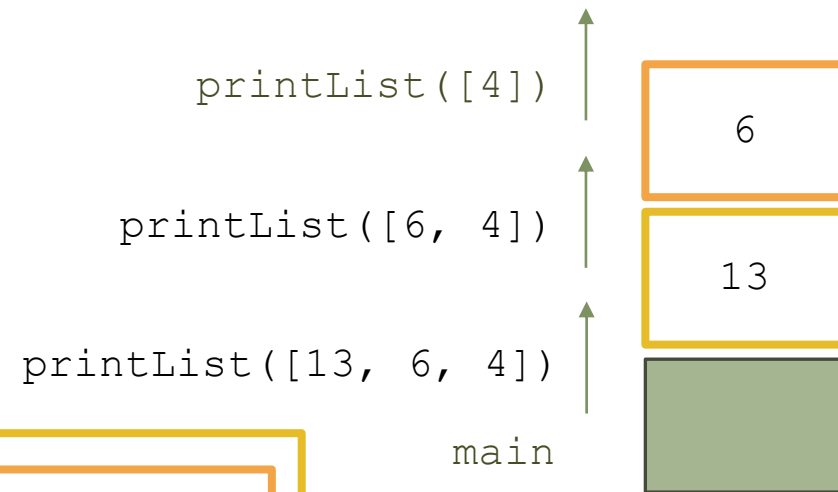
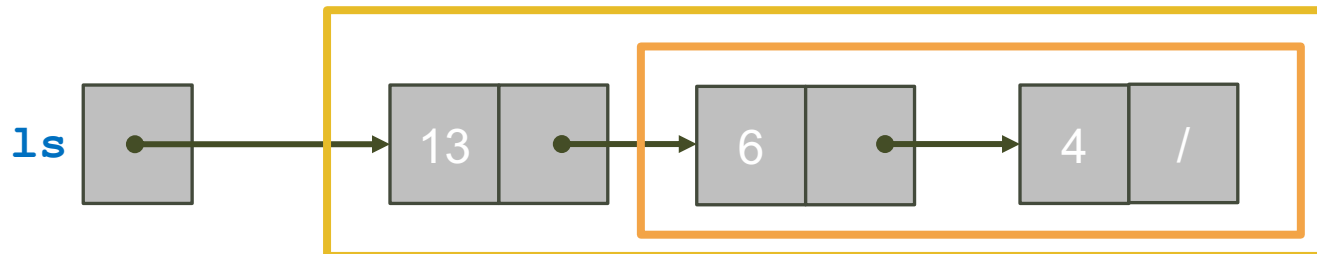


Stampa: 13 6

Simulazione: printList()



```
void printList(List L) {  
    // Caso base  
    if (L == NULL) return;  
  
    //Caso induttivo  
    printf("%d ", L->data);  
    return printList(L->next);  
}
```

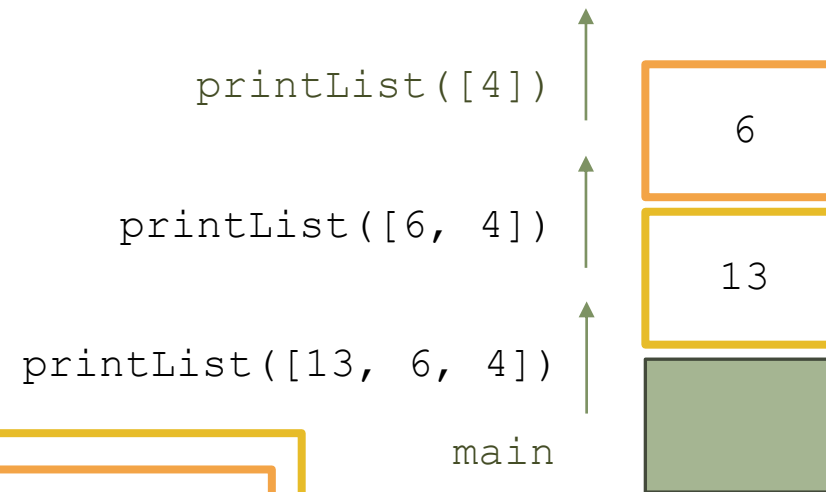
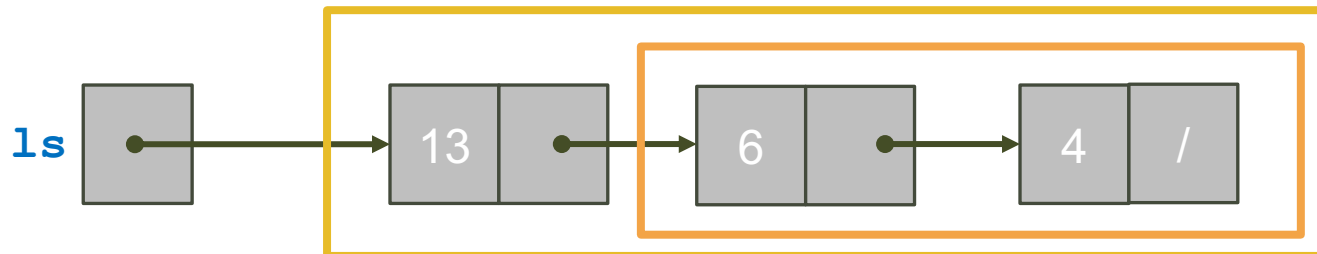


Stampa: 13 6

Simulazione: printList()



```
void printList(List L) {  
    // Caso base  
    if (L == NULL) return;  
  
    //Caso induttivo  
    printf("%d ", L->data);  
    return printList(L->next);  
}
```

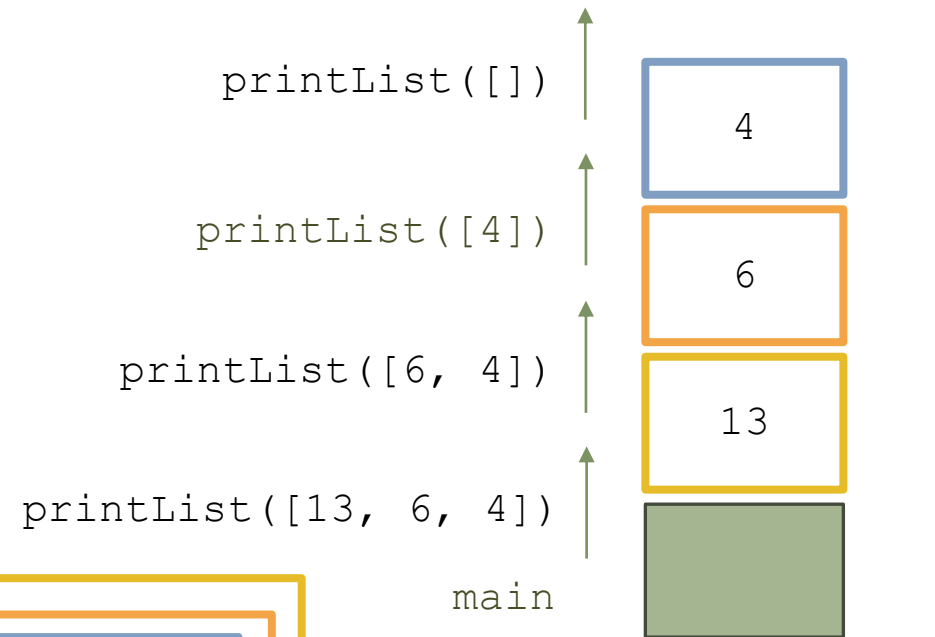
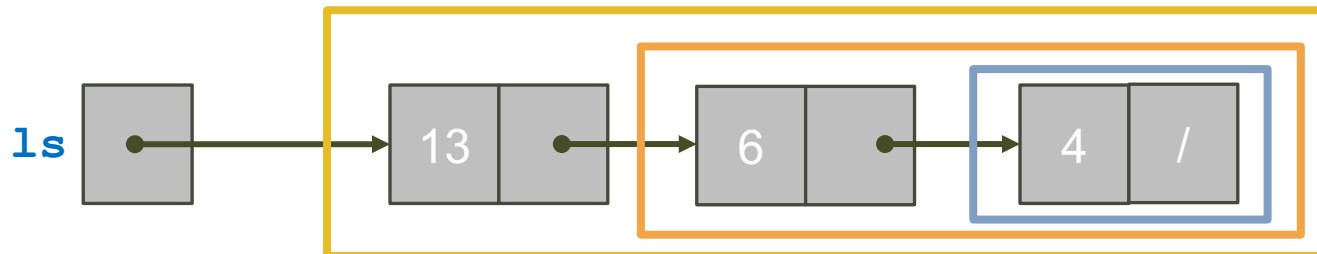


Stampa: 13 6 4

Simulazione: printList()



```
void printList(List L) {  
    // Caso base  
    if (L == NULL) return;  
  
    //Caso induttivo  
    printf("%d ", L->data);  
    return printList(L->next);  
}
```

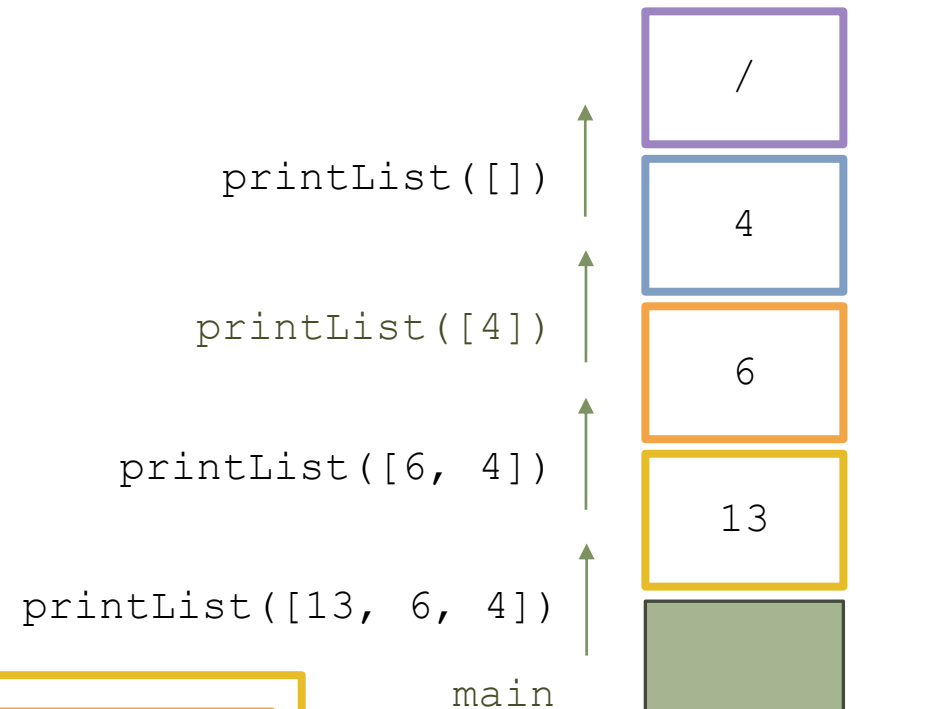
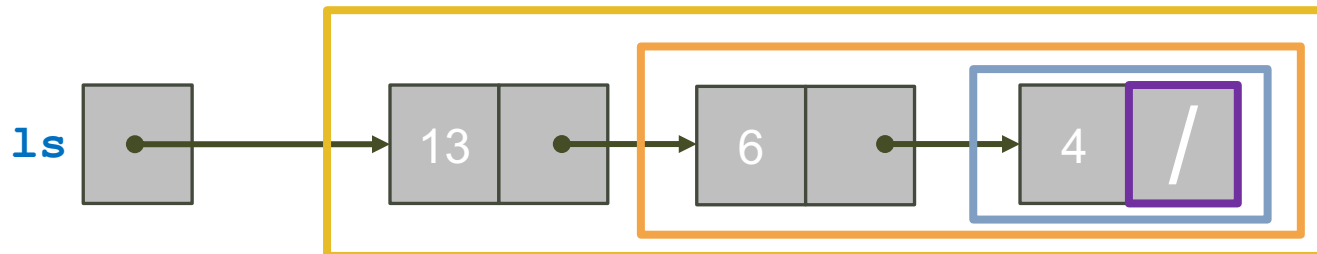


Stampa: 13 6 4

Simulazione: printList()



```
void printList(List L) {  
    // Caso base  
    if (L == NULL) return;  
  
    //Caso induttivo  
    printf("%d ", L->data);  
    return printList(L->next);  
}
```

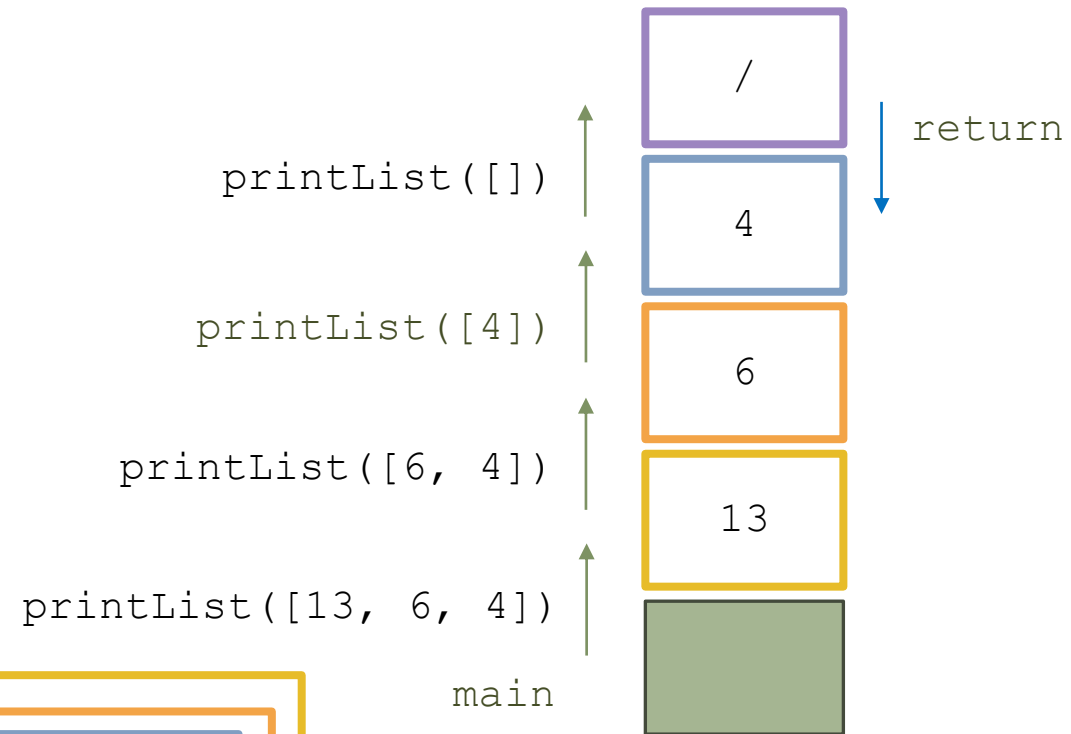
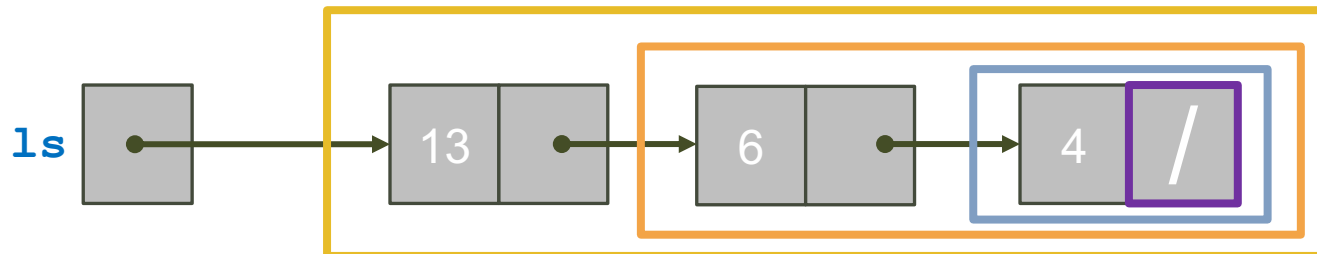


Stampa: 13 6 4

Simulazione: printList()



```
void printList(List L) {  
    // Caso base  
    if (L == NULL) return;  
  
    //Caso induttivo  
    printf("%d ", L->data);  
    return printList(L->next);  
}
```

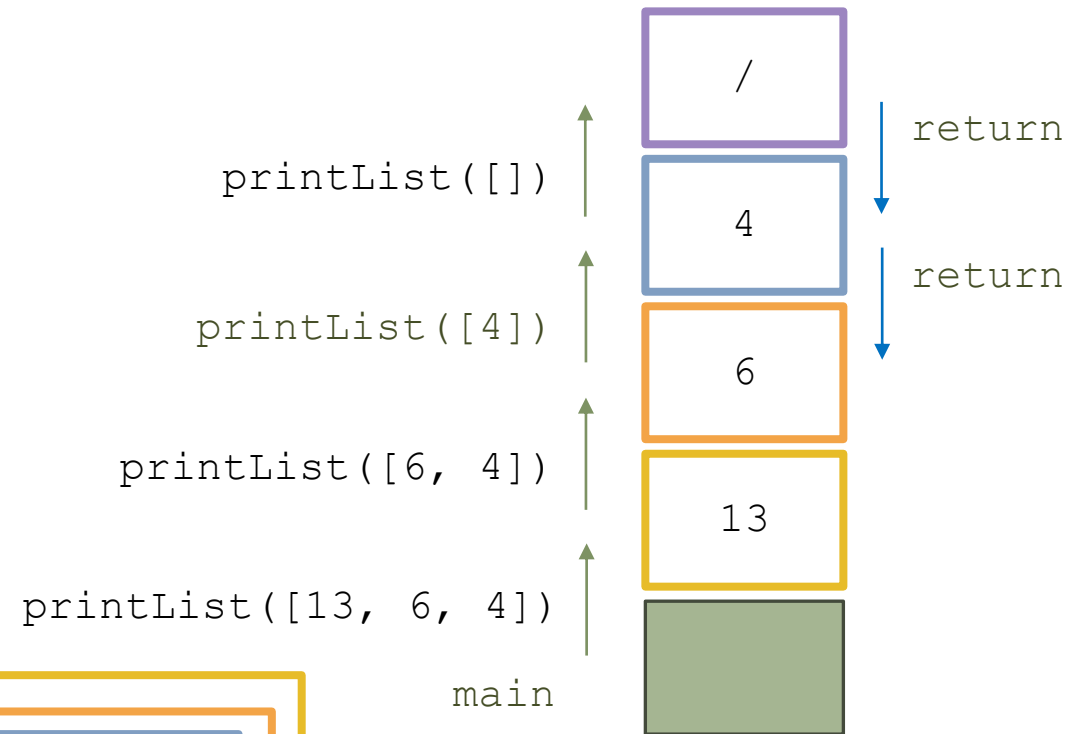
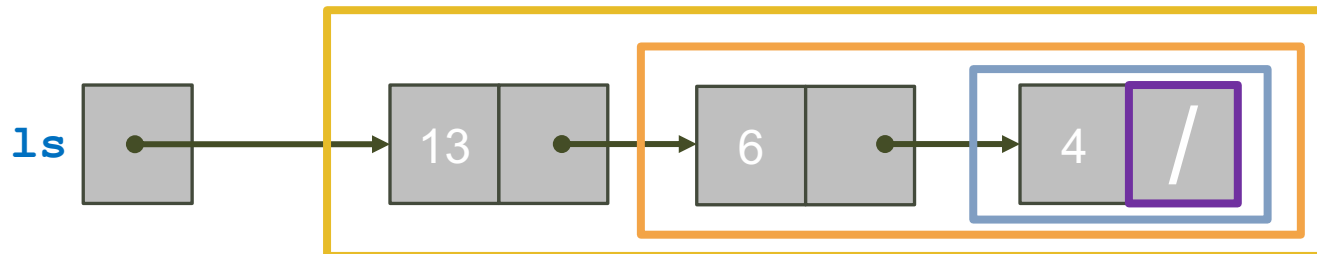


Stampa: 13 6 4

Simulazione: printList()



```
void printList(List L) {  
    // Caso base  
    if (L == NULL) return;  
  
    //Caso induttivo  
    printf("%d ", L->data);  
    return printList(L->next);  
}
```

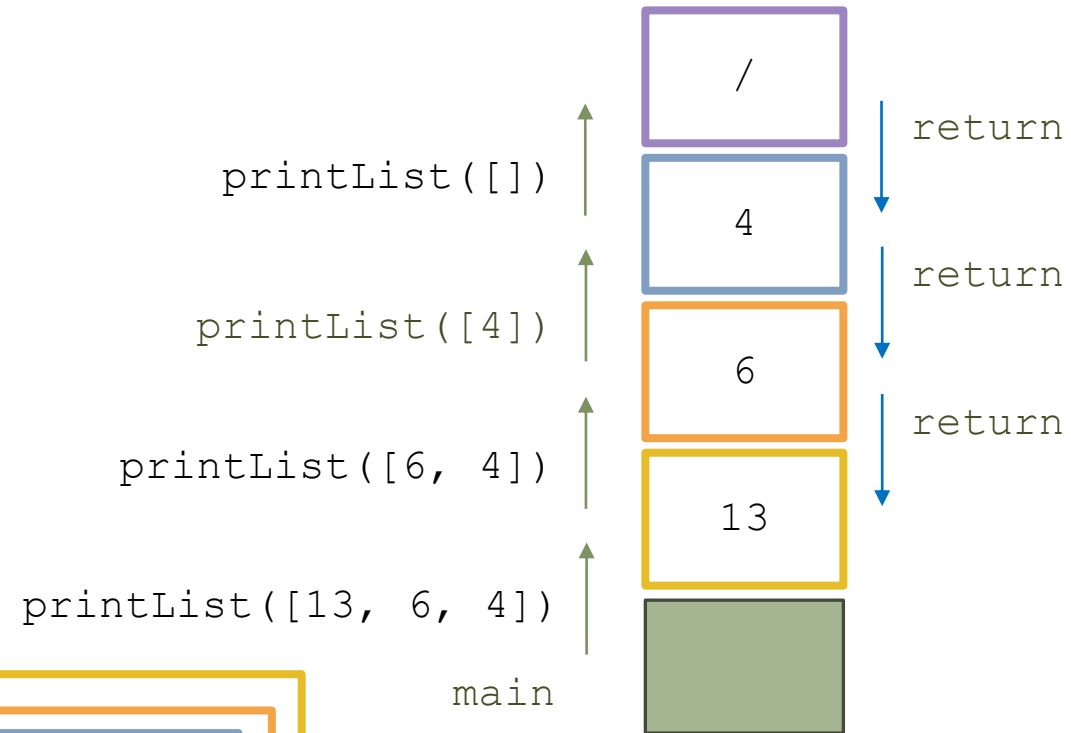
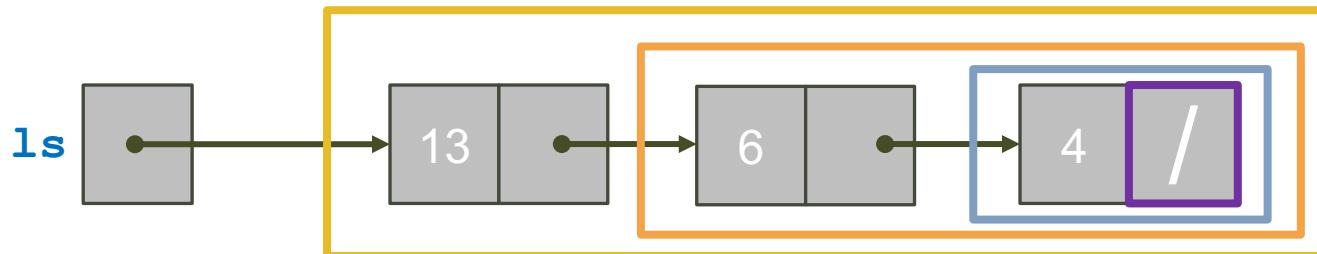


Stampa: 13 6 4

Simulazione: printList()

```
void printList(List L) {
    // Caso base
    if (L == NULL) return;

    //Caso induttivo
    printf("%d ", L->data);
    return printList(L->next);
}
```

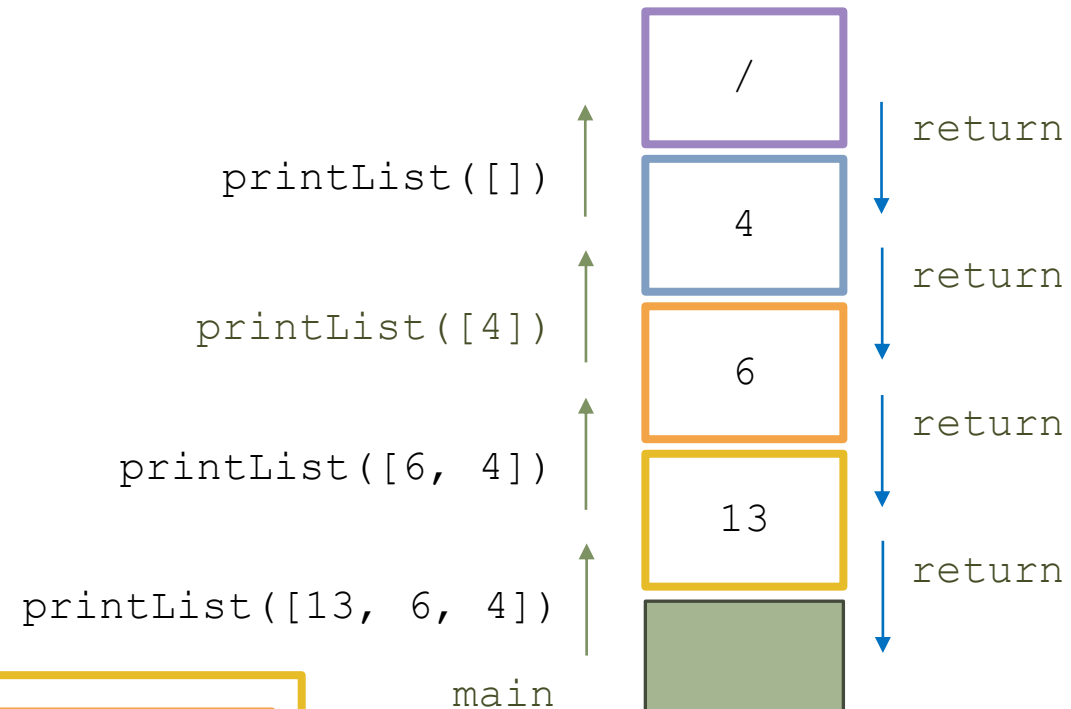
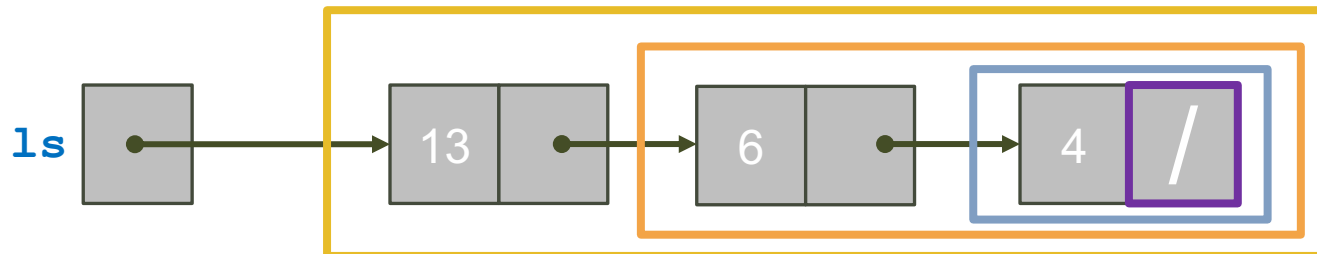


Stampa: 13 6 4

Simulazione: printList()



```
void printList(List L) {  
    // Caso base  
    if (L == NULL) return;  
  
    //Caso induttivo  
    printf("%d ", L->data);  
    return printList(L->next);  
}
```



Stampa: 13 6 4

printList() : Variante

- Stampa elementi lista
 - Variante
 - Semplice preferenza

```
void printList(List L) {  
    if (L != NULL) {  
        printf("%d\n", L->data);  
        return printList(L->next);  
    }  
}
```



printList() : Variante 2

- Ricorsione in testa
 - Cosa succede?
 - Stampa in ordine inverso

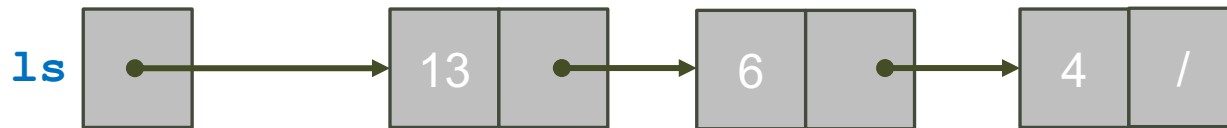
```
void printList(List L) {  
    // Caso base  
    if (L == NULL) return;  
  
    //Caso induttivo  
    printList(L->next);  
    printf("%d ", L->data);  
}
```



Simulazione: `printList()` Variante 2



```
void printList(List L) {  
    // Caso base  
    if (L == NULL) return;  
  
    //Caso induttivo  
    printList(L->next);  
    printf("%d ", L->data);  
}
```

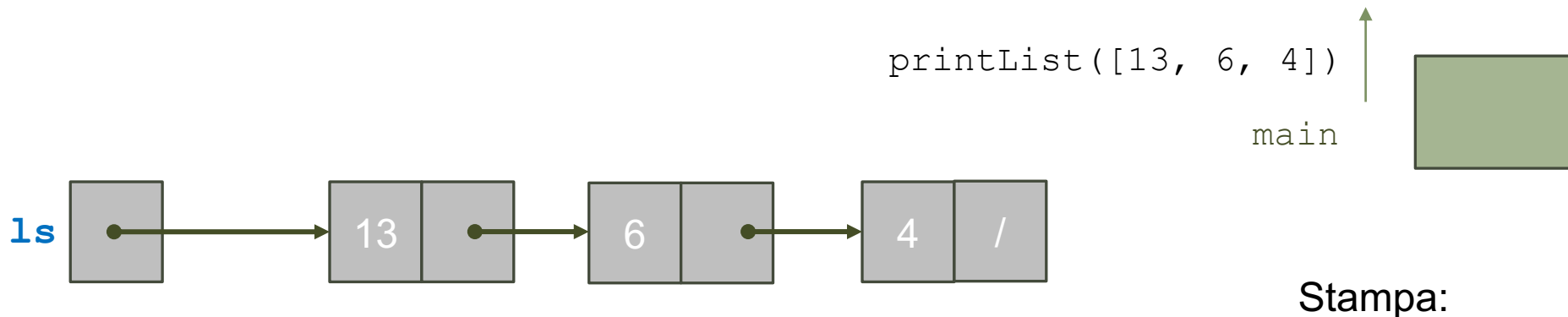


Stampa:

Simulazione: `printList()` Variante 2



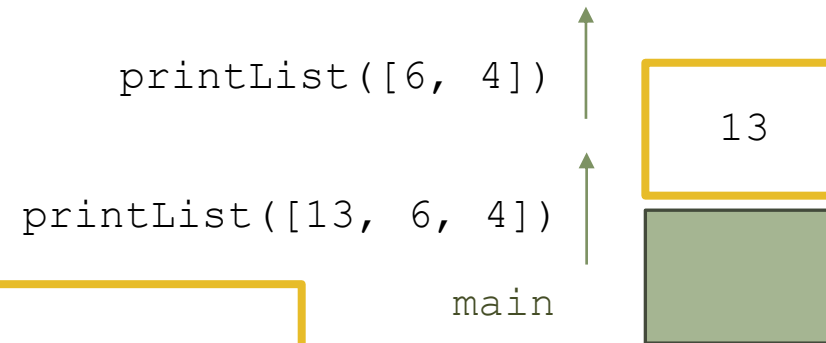
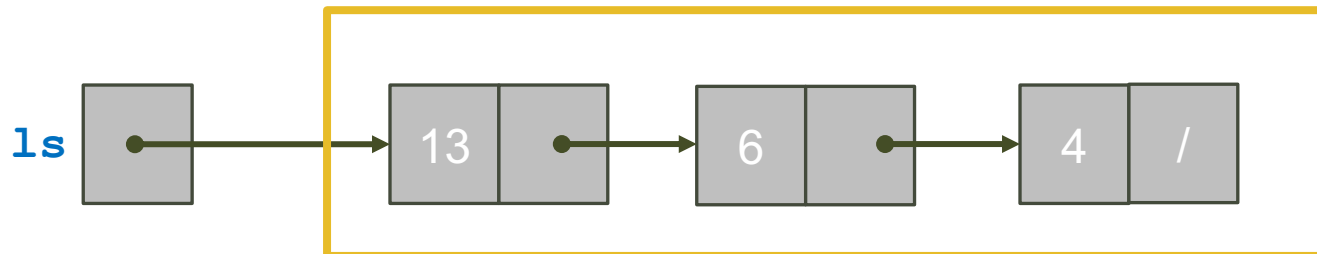
```
void printList(List L) {  
    // Caso base  
    if (L == NULL) return;  
  
    //Caso induttivo  
    printList(L->next);  
    printf("%d ", L->data);  
}
```



Simulazione: `printList()` Variante 2



```
void printList(List L) {  
    // Caso base  
    if (L == NULL) return;  
  
    //Caso induttivo  
    printList(L->next);  
    printf("%d ", L->data);  
}
```

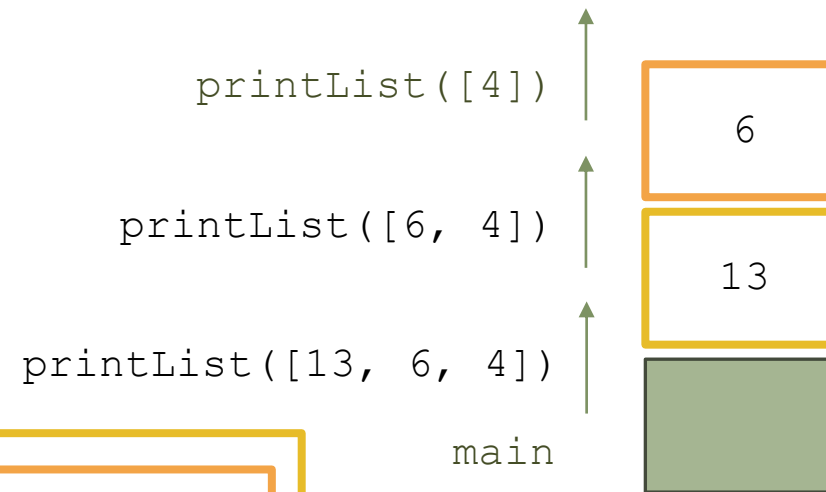
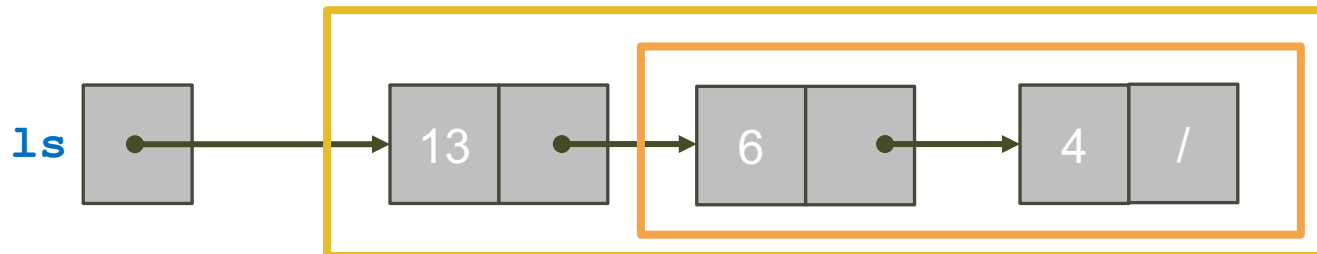


Stampa:

Simulazione: `printList()` Variante 2



```
void printList(List L) {  
    // Caso base  
    if (L == NULL) return;  
  
    //Caso induttivo  
    printList(L->next);  
    printf("%d ", L->data);  
}
```

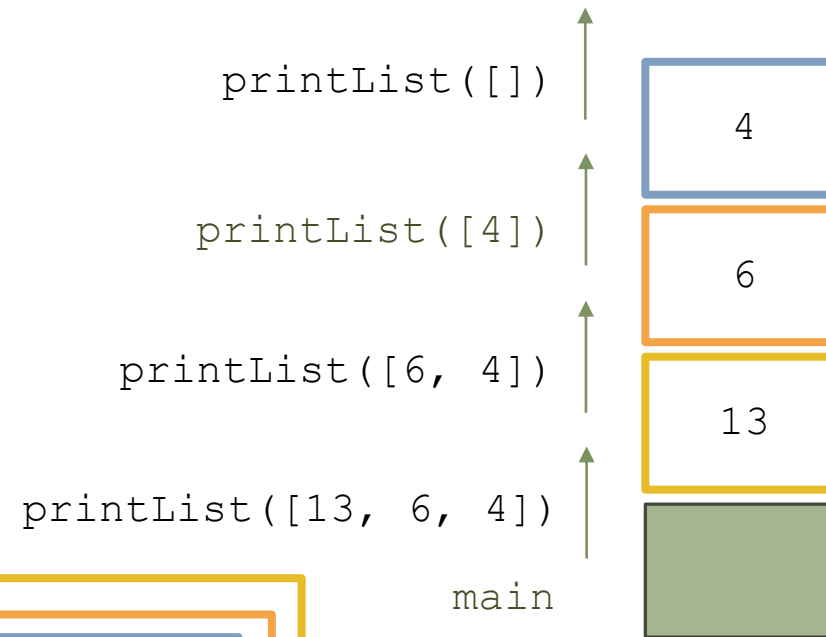
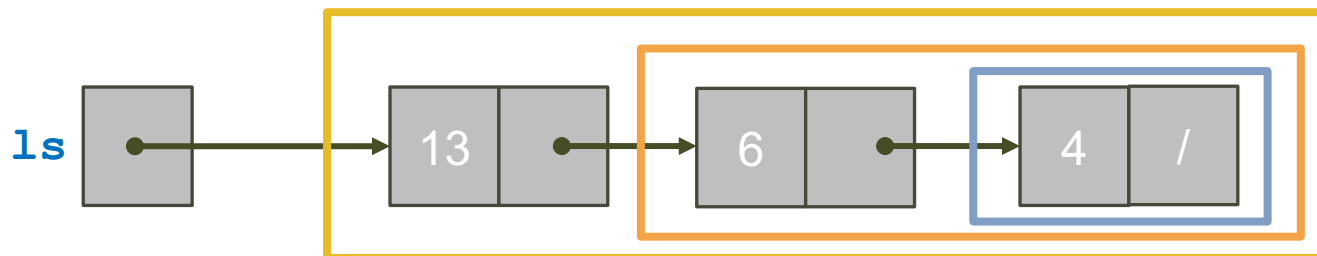


Stampa:

Simulazione: `printList()` Variante 2



```
void printList(List L) {  
    // Caso base  
    if (L == NULL) return;  
  
    //Caso induttivo  
    printList(L->next);  
    printf("%d ", L->data);  
}
```

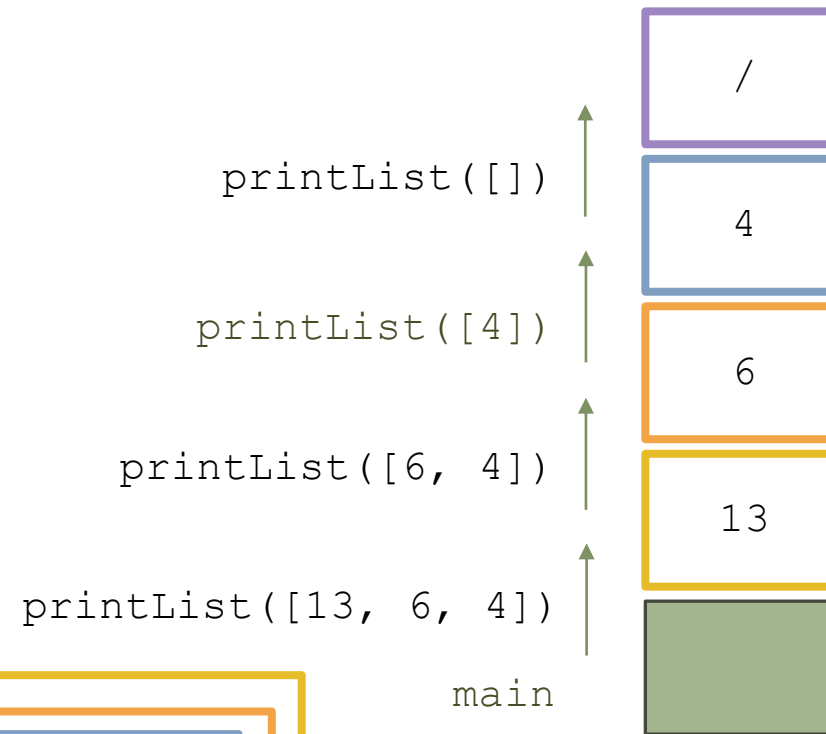
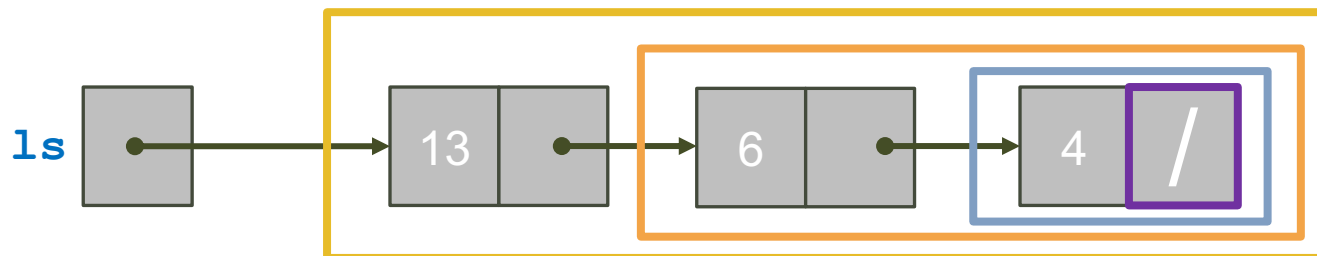


Stampa:

Simulazione: `printList()` Variante 2



```
void printList(List L) {  
    // Caso base  
    if (L == NULL) return;  
  
    //Caso induttivo  
    printList(L->next);  
    printf("%d ", L->data);  
}
```

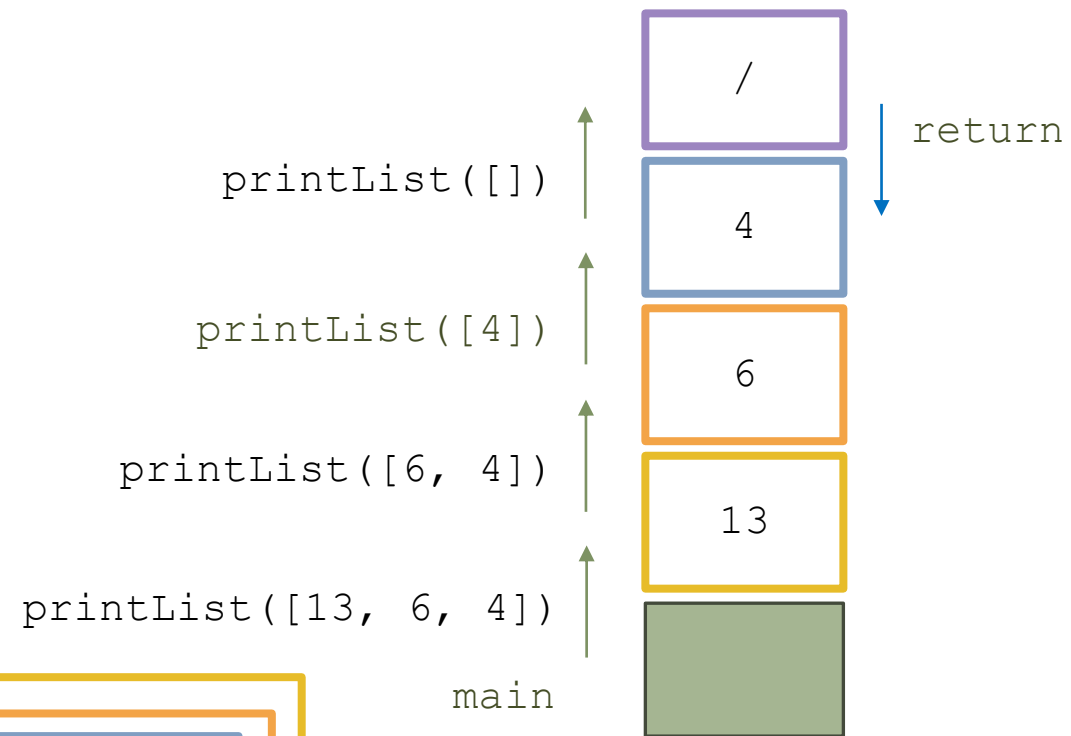
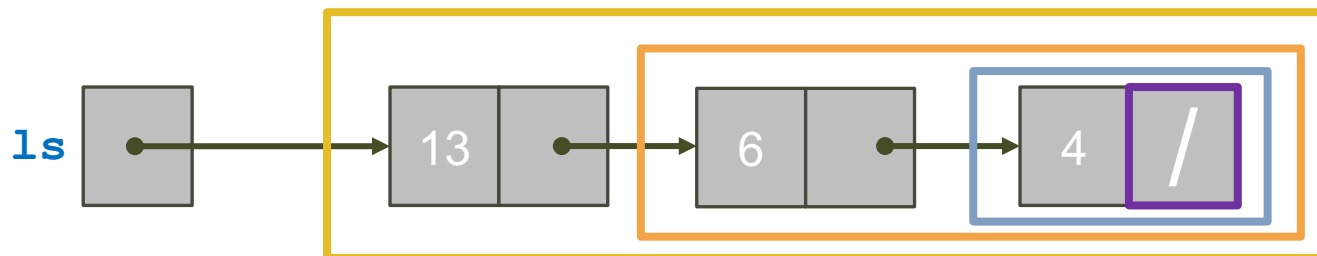


Stampa:

Simulazione: printList() Variante 2



```
void printList(List L) {  
    // Caso base  
    if (L == NULL) return;  
  
    //Caso induttivo  
    printList(L->next);  
    printf("%d ", L->data);  
}
```

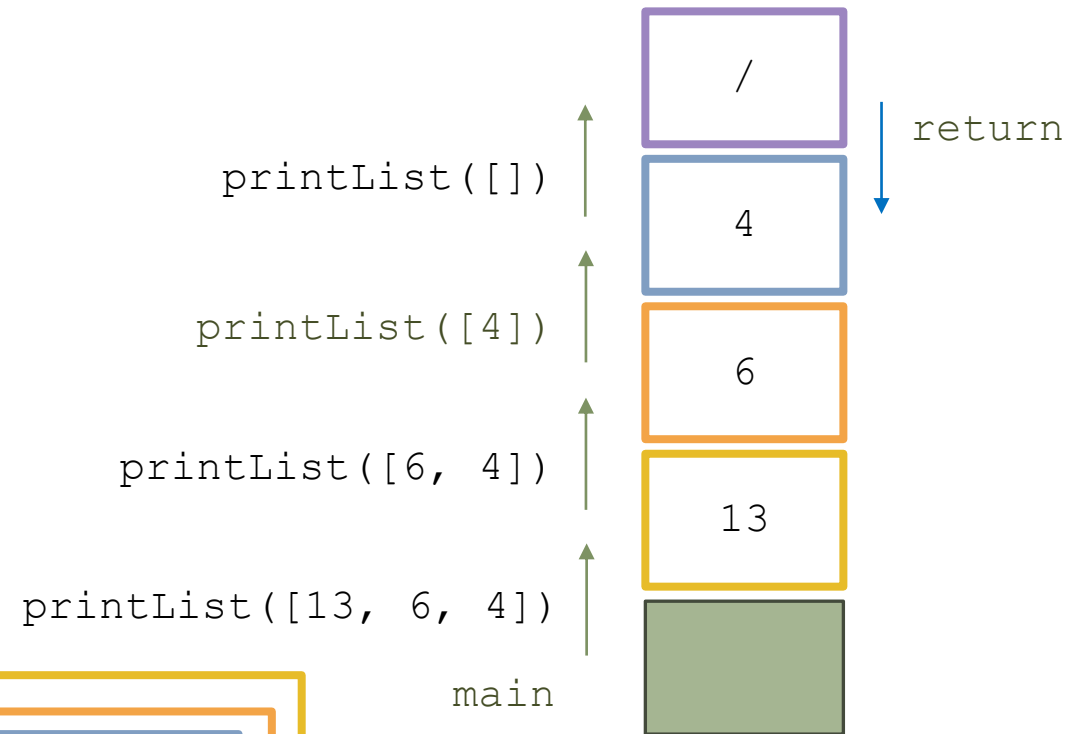
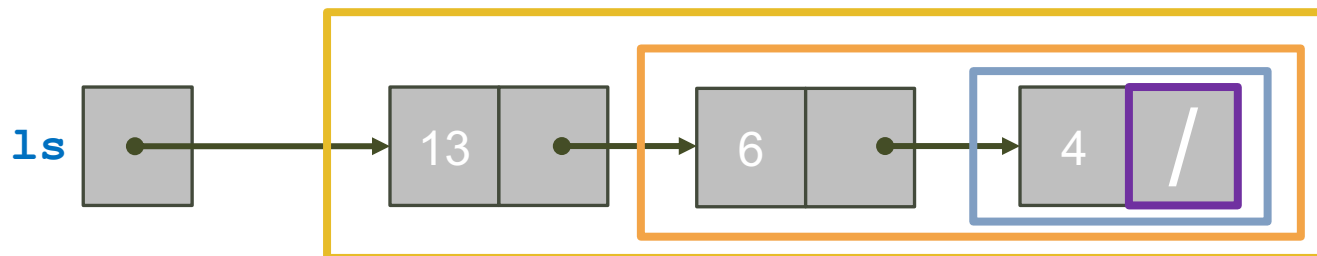


Stampa:

Simulazione: printList() Variante 2



```
void printList(List L) {  
    // Caso base  
    if (L == NULL) return;  
  
    //Caso induttivo  
    printList(L->next);  
    printf("%d ", L->data);  
}
```

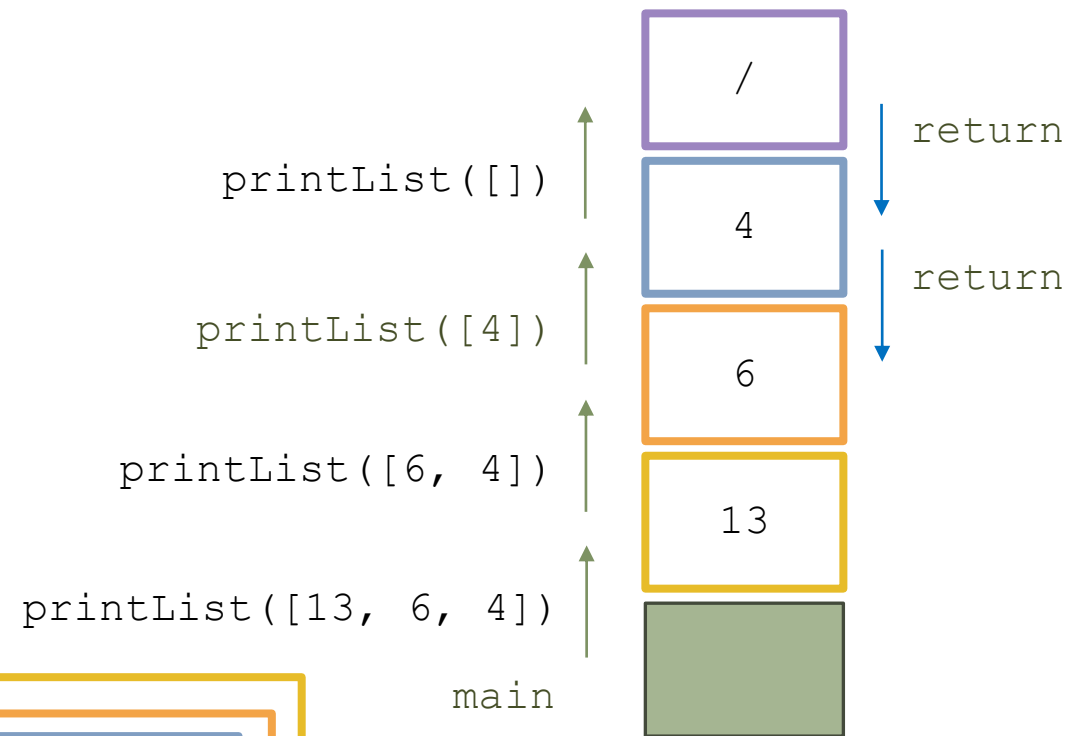
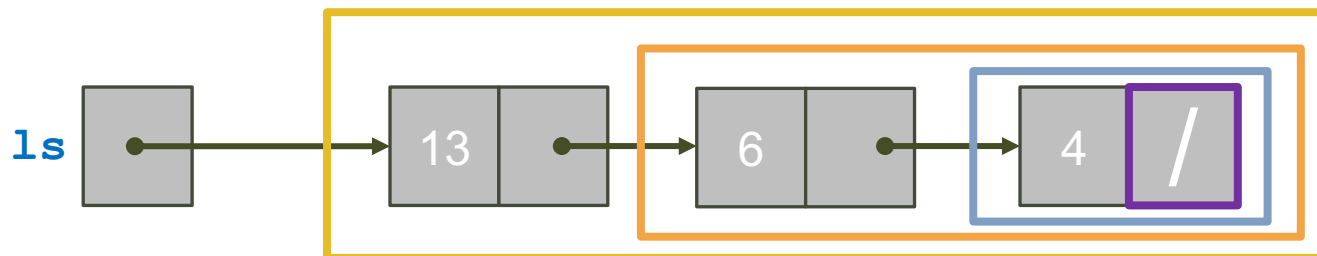


Stampa: 4

Simulazione: `printList()` Variante 2



```
void printList(List L) {  
    // Caso base  
    if (L == NULL) return;  
  
    //Caso induttivo  
    printList(L->next);  
    printf("%d ", L->data);  
}
```

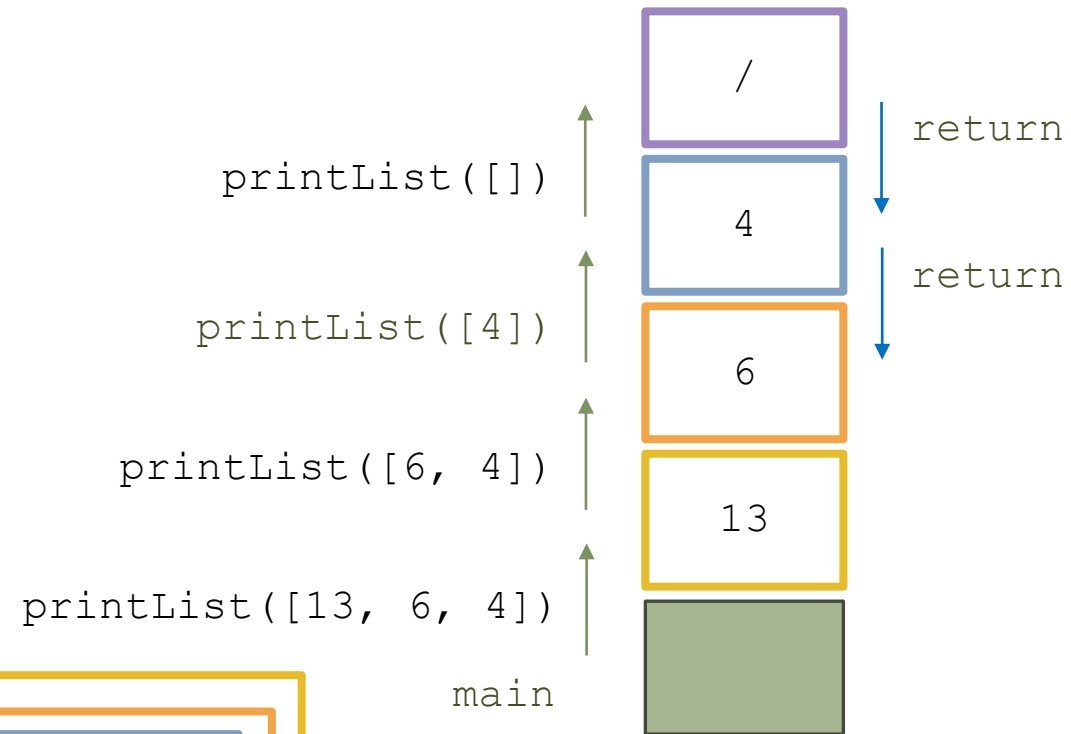
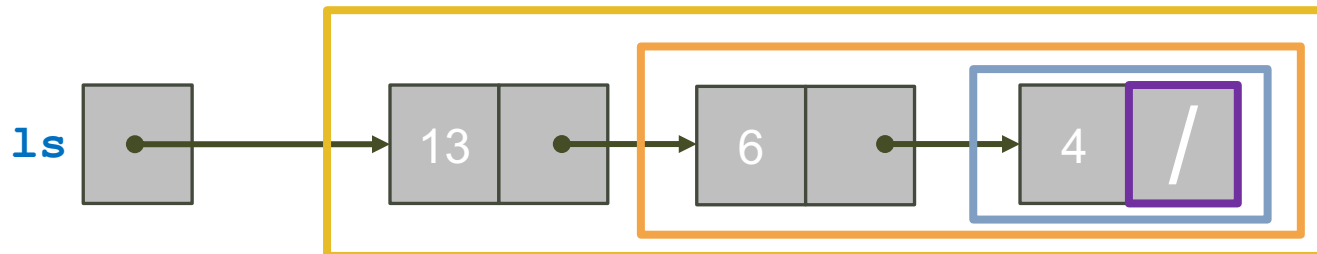


Stampa: 4

Simulazione: `printList()` Variante 2



```
void printList(List L) {  
    // Caso base  
    if (L == NULL) return;  
  
    //Caso induttivo  
    printList(L->next);  
    printf("%d ", L->data);  
}
```

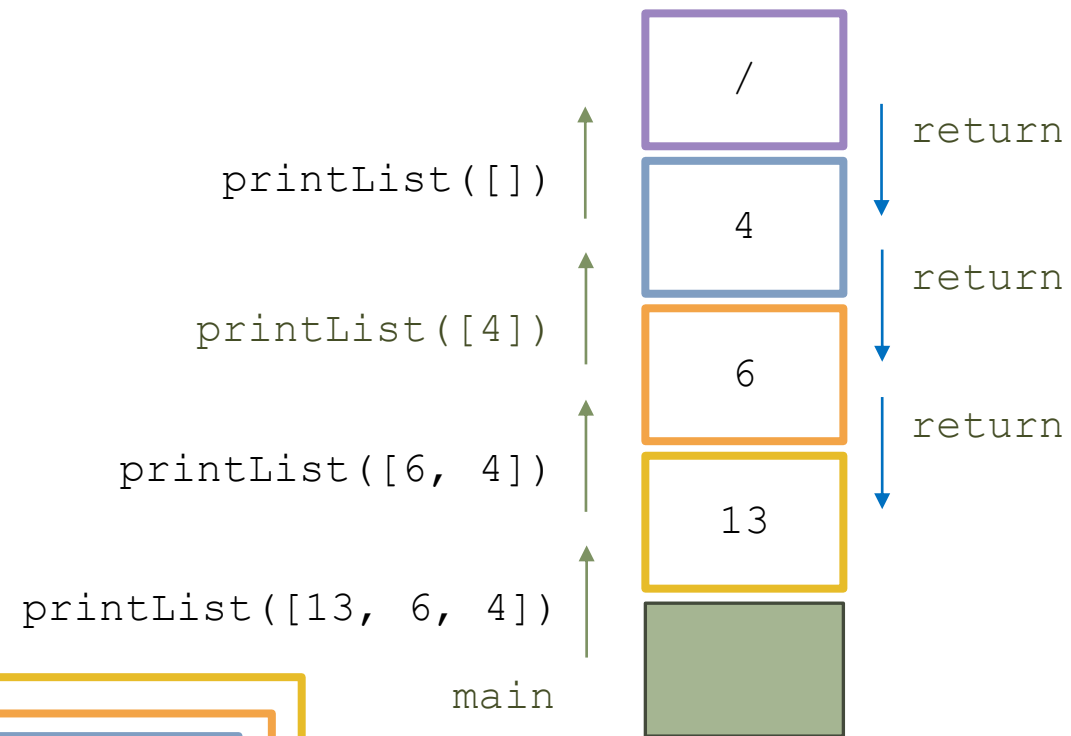
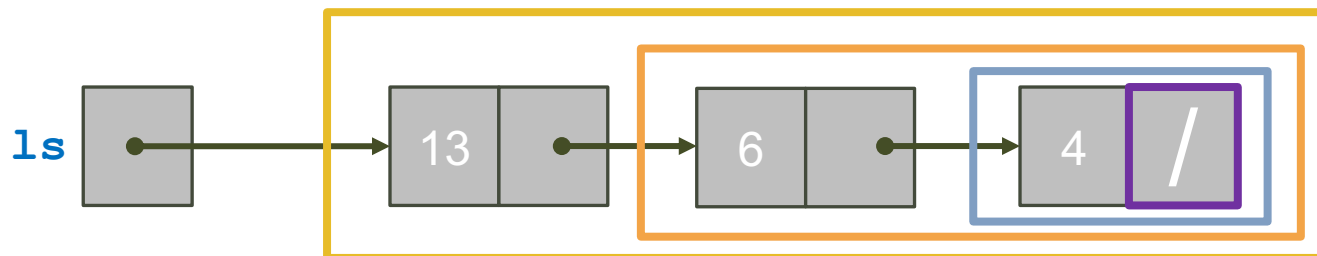


Stampa: 4 6

Simulazione: `printList()` Variante 2



```
void printList(List L) {  
    // Caso base  
    if (L == NULL) return;  
  
    //Caso induttivo  
    printList(L->next);  
    printf("%d ", L->data);  
}
```

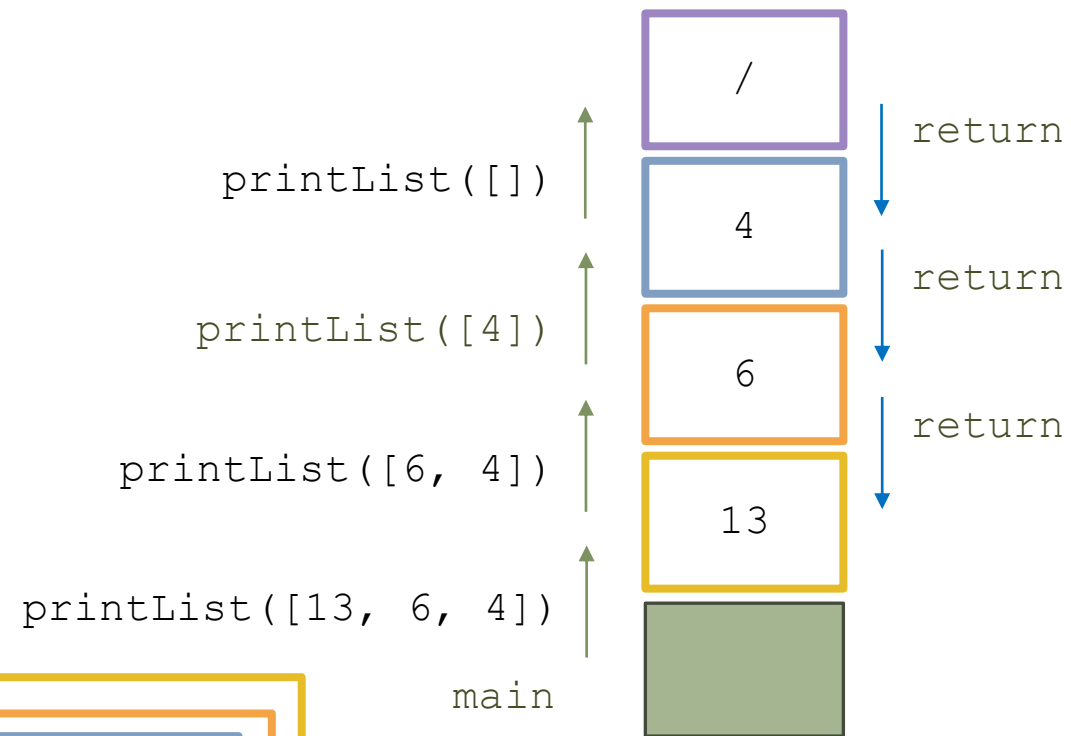
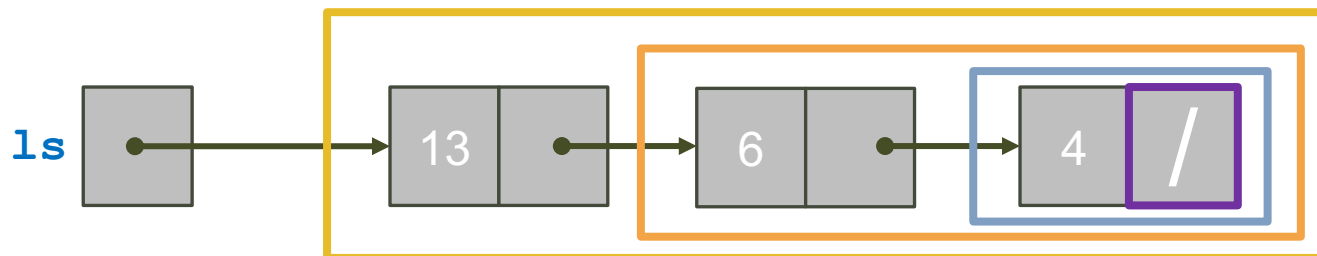


Stampa: 4 6

Simulazione: printList() Variante 2



```
void printList(List L) {  
    // Caso base  
    if (L == NULL) return;  
  
    //Caso induttivo  
    printList(L->next);  
    printf("%d ", L->data);  
}
```

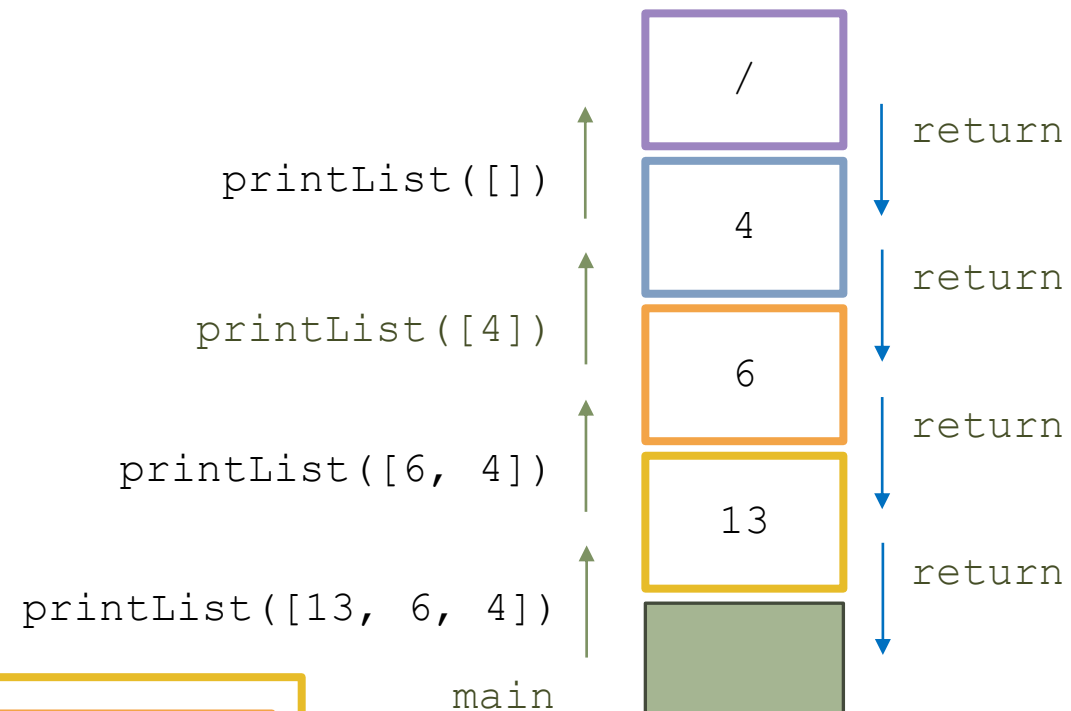
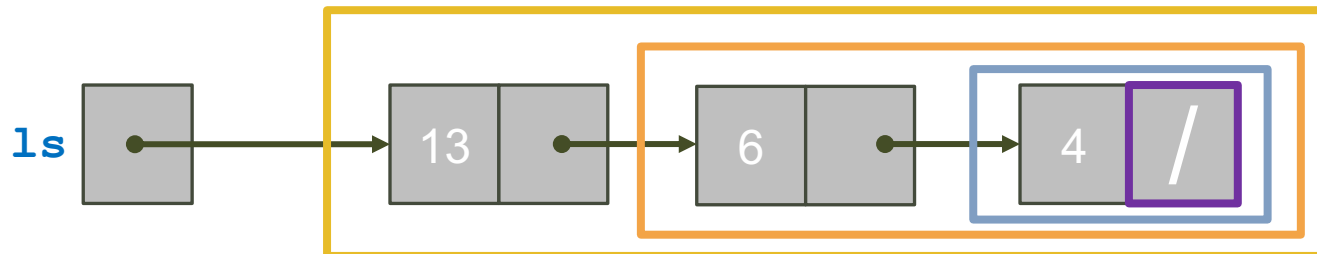


Stampa: 4 6 13

Simulazione: `printList()` Variante 2



```
void printList(List L) {  
    // Caso base  
    if (L == NULL) return;  
  
    //Caso induttivo  
    printList(L->next);  
    printf("%d ", L->data);  
}
```



Stampa: 4 6 13

Esempio: `sumIntList()`



- Calcolo somma dei valori nei nodi di una lista linkata

- Procedimento?

-

-

-

-

-

-



```
int sumIntList(List L) {  
  
  
  
}
```

Esempio: `sumIntList()`

- Calcolo somma dei valori nei nodi di una lista linkata
 - Procedimento?
 - Percorro la lista accumulando i valori via via incontrati
 -
 -
 -
 -
 -

```
int sumIntList(List L) {  
  
}  

```



Esempio: `sumIntList()`

- Calcolo somma dei valori nei nodi di una lista linkata
 - Procedimento?
 - Percorro la lista accumulando i valori via via incontrati
 - Caso base?
 -
 -
 -
 -

```
int sumIntList(List L) {  
  
}
```



Esempio: `sumIntList()`

- Calcolo somma dei valori nei nodi di una lista linkata
 - Procedimento?
 - Percorro la lista accumulando i valori via via incontrati
 - Caso base?
 - Lista vuota
 -
 -
 -

```
int sumIntList(List L) {  
  
  
  
}
```



Esempio: `sumIntList()`

- Calcolo somma dei valori nei nodi di una lista linkata
 - Procedimento?
 - Percorro la lista accumulando i valori via via incontrati
 - Caso base?
 - Lista vuota
 -
 -
 -

```
int sumIntList(List L) {  
    if (L == NULL)  
  
}
```



Esempio: `sumIntList()`

- Calcolo somma dei valori nei nodi di una lista linkata
 - Procedimento?
 - Percorro la lista accumulando i valori via via incontrati
 - Caso base?
 - Lista vuota
 - Anche questo caso produce un valore: quale?
 -
 -

```
int sumIntList(List L) {  
    if (L == NULL)  
  
}
```



Esempio: `sumIntList()`

- Calcolo somma dei valori nei nodi di una lista linkata
 - Procedimento?
 - Percorro la lista accumulando i valori via via incontrati
 - Caso base?
 - Lista vuota
 - Anche questo caso produce un valore: quale? 0
 -
 -

```
int sumIntList(List L) {  
    if (L == NULL)  
  
}
```



Esempio: `sumIntList()`

- Calcolo somma dei valori nei nodi di una lista linkata
 - Procedimento?
 - Percorro la lista accumulando i valori via via incontrati
 - Caso base?
 - Lista vuota
 - Anche questo caso produce un valore: quale? 0
 -
 -

```
int sumIntList(List L) {  
    if (L == NULL)  
        return 0;  
  
}
```

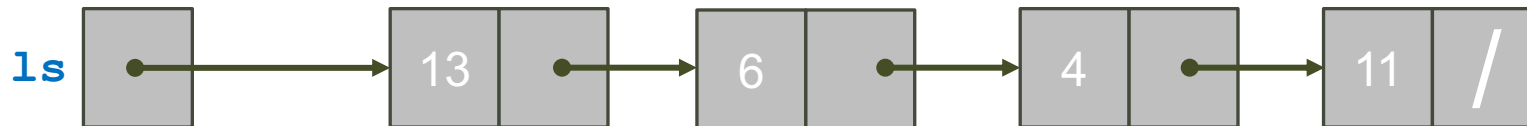


Esempio: `sumIntList()`



- Calcolo somma dei valori nei nodi di una lista linkata
 - Procedimento?
 - Percorro la lista accumulando i valori via via incontrati
 - Caso base?
 - Lista vuota
 - Anche questo caso produce un valore: quale? 0
 - Calcolo ricorsivo?
 -

```
int sumIntList(List L) {  
    if (L == NULL)  
        return 0;  
  
}
```



Esempio: `sumIntList()`



- Calcolo somma dei valori nei nodi di una lista linkata
 - Procedimento?
 - Percorro la lista accumulando i valori via via incontrati
 - Caso base?
 - Lista vuota
 - Anche questo caso produce un valore: quale? 0
 - Calcolo ricorsivo?
 - Somma: valore nodo corrente + somma lista che segue

```
int sumIntList(List L) {  
    if (L == NULL)  
        return 0;  
  
}
```



Esempio: `sumIntList()`

- Calcolo somma dei valori nei nodi di una lista linkata
 - Procedimento?
 - Percorro la lista accumulando i valori via via incontrati
 - Caso base?
 - Lista vuota
 - Anche questo caso produce un valore: quale? 0
 - Calcolo ricorsivo?
 - Somma: valore nodo corrente + somma lista che segue

```
int sumIntList(List L) {  
    if (L == NULL)  
        return 0;  
  
    return L->val  
        + sumIntList(L->next);  
}
```

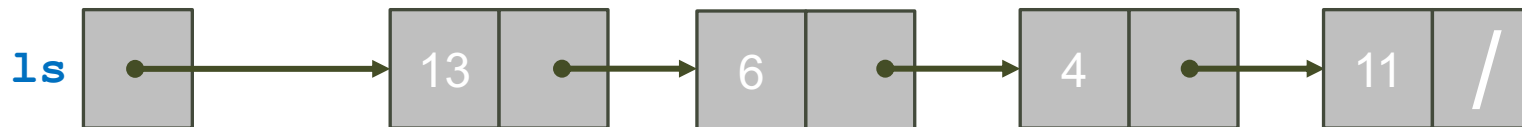


Esempio: `sumIntList()`

- Calcolo somma dei valori nei nodi di una lista linkata
 - Procedimento?
 - Percorro la lista accumulando i valori via via incontrati
 - Caso base?
 - Lista vuota
 - Anche questo caso produce un valore: quale? 0
 - Calcolo ricorsivo?
 - Somma: valore nodo corrente + somma lista che segue

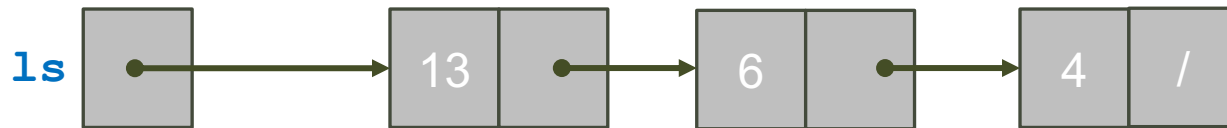
```
int sumIntList(List L) {  
    if (L == NULL)  
        return 0;  
  
    return L->val  
        + sumIntList(L->next);  
}
```

Progresso



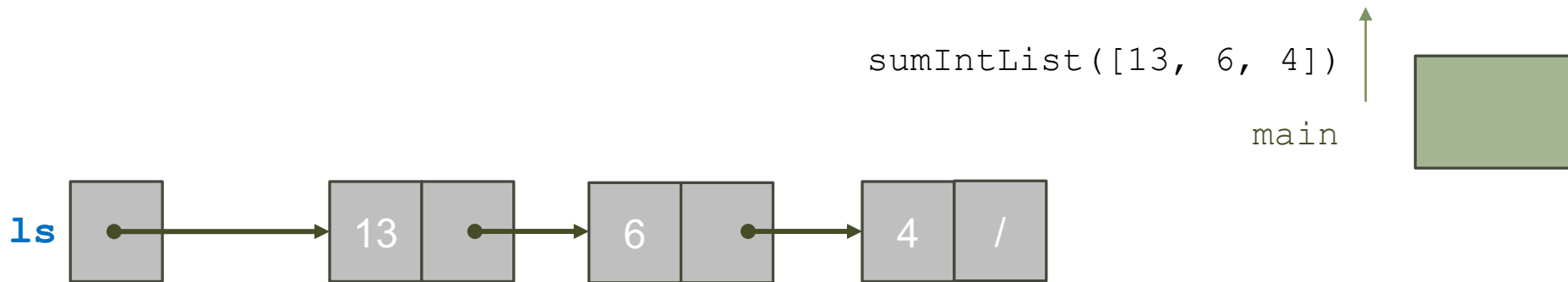
Simulazione: `sumIntList()`

```
int sumIntList(List L) {  
    if (L == NULL)  
        return 0;  
  
    return L->val  
        + sumIntList(L->next);  
}
```



Simulazione: sumIntList()

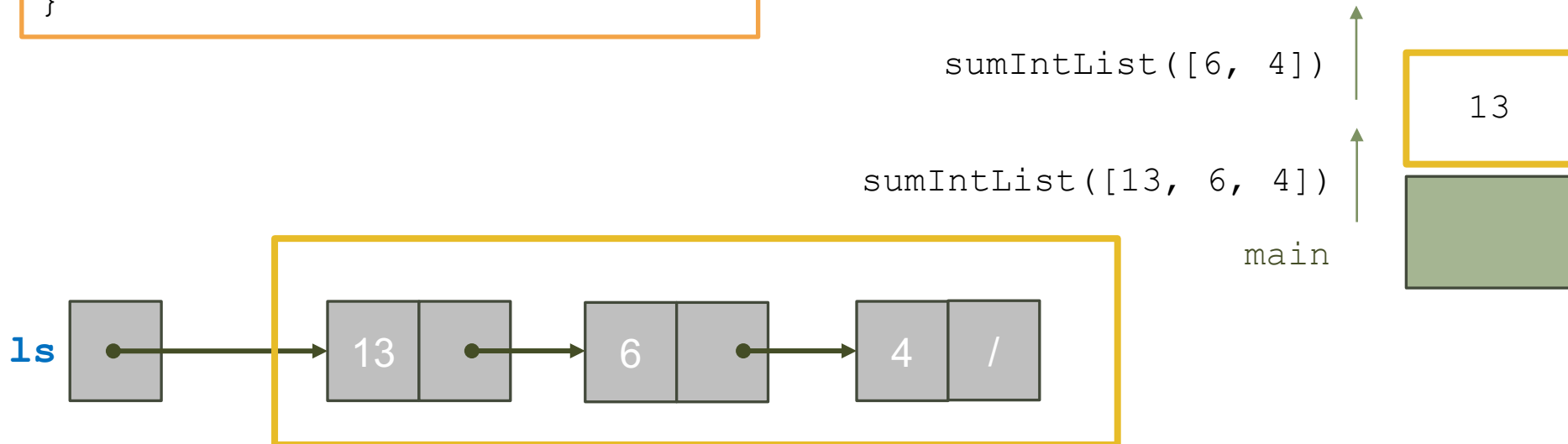
```
int sumIntList(List L) {  
    if (L == NULL)  
        return 0;  
  
    return L->val  
        + sumIntList(L->next);  
}
```



Simulazione: `sumIntList()`

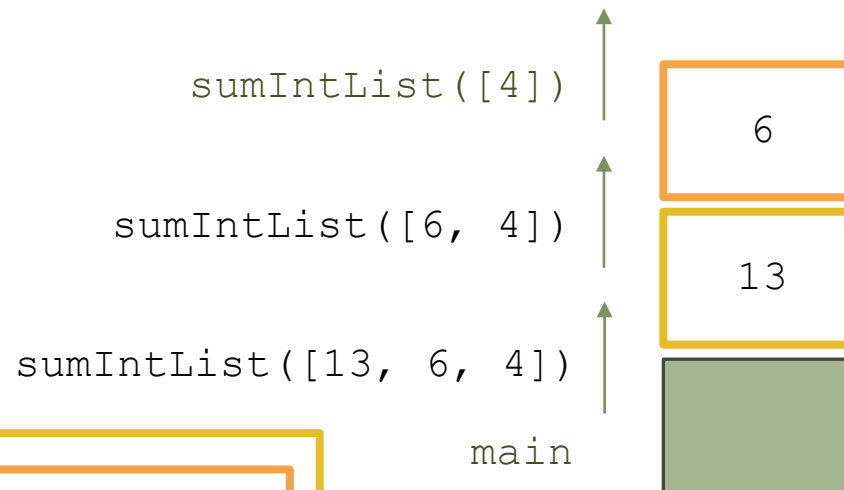
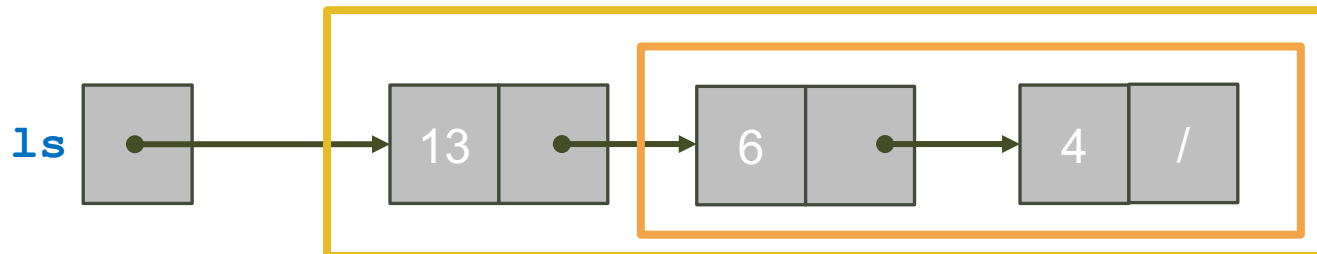
```
int sumIntList(List L) {
    if (L == NULL)
        return 0;

    return L->val
        + sumIntList(L->next);
}
```



Simulazione: `sumIntList()`

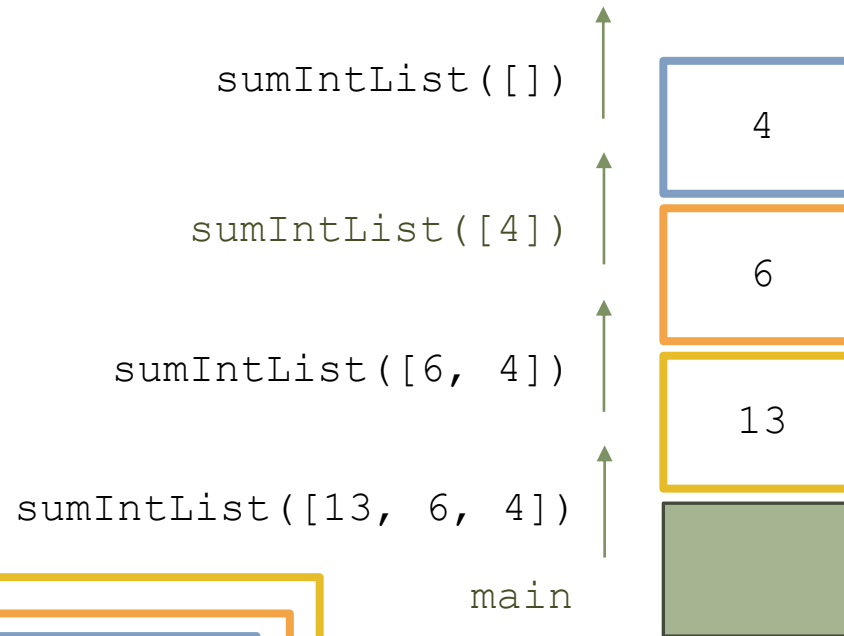
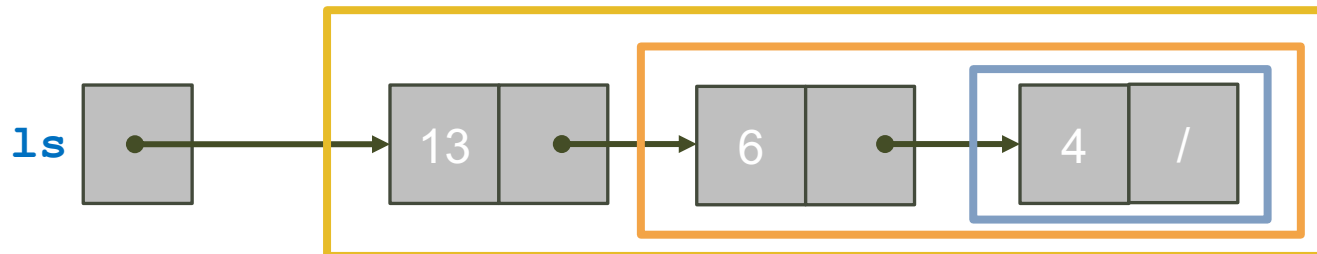
```
int sumIntList(List L) {  
    if (L == NULL)  
        return 0;  
  
    return L->val  
        + sumIntList(L->next);  
}
```



Simulazione: `sumIntList()`

```
int sumIntList(List L) {
    if (L == NULL)
        return 0;

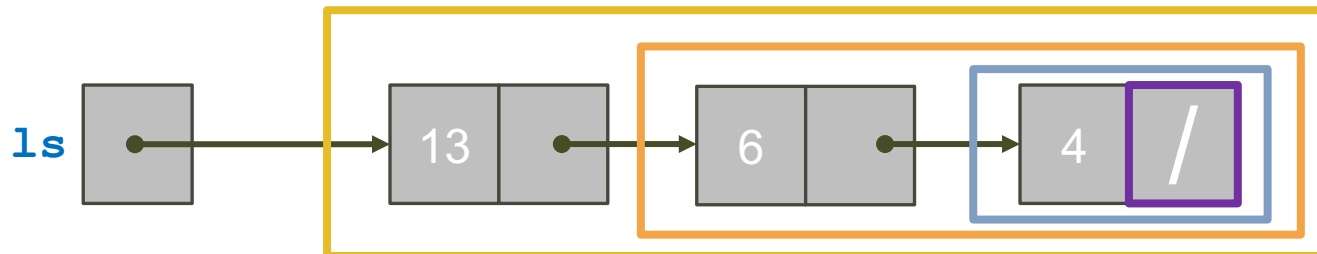
    return L->val
        + sumIntList(L->next);
}
```



Simulazione: `sumIntList()`



```
int sumIntList(List L) {  
    if (L == NULL)  
        return 0;  
  
    return L->val  
        + sumIntList(L->next);  
}
```



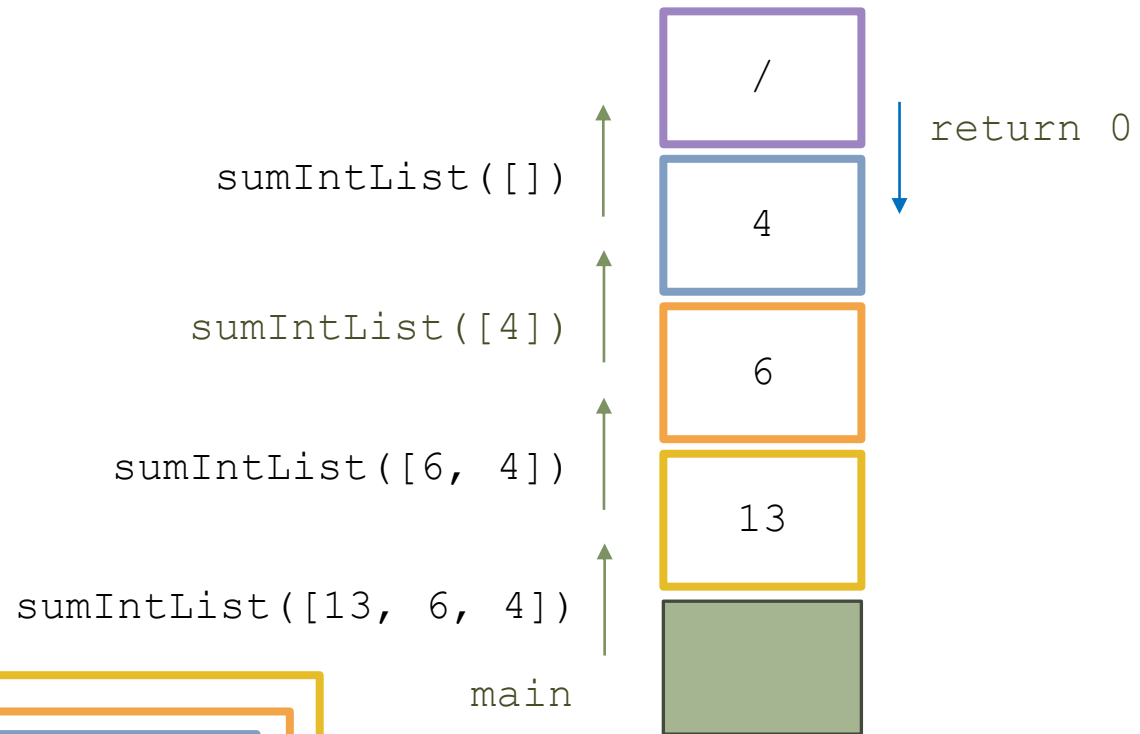
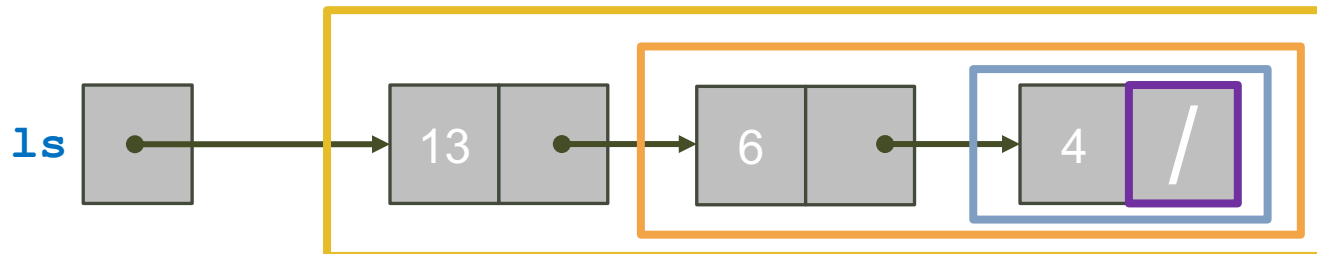
`sumIntList([])`
`sumIntList([4])`
`sumIntList([6, 4])`
`sumIntList([13, 6, 4])`
main



Simulazione: sumIntList()

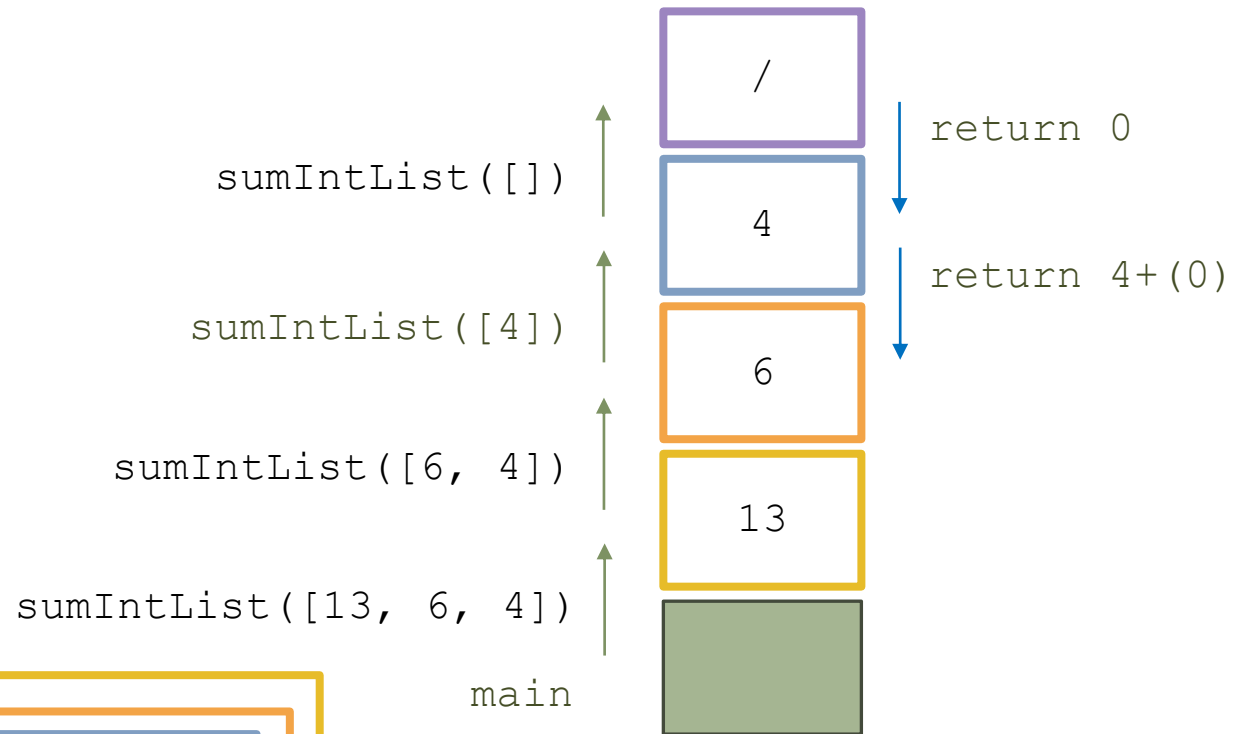
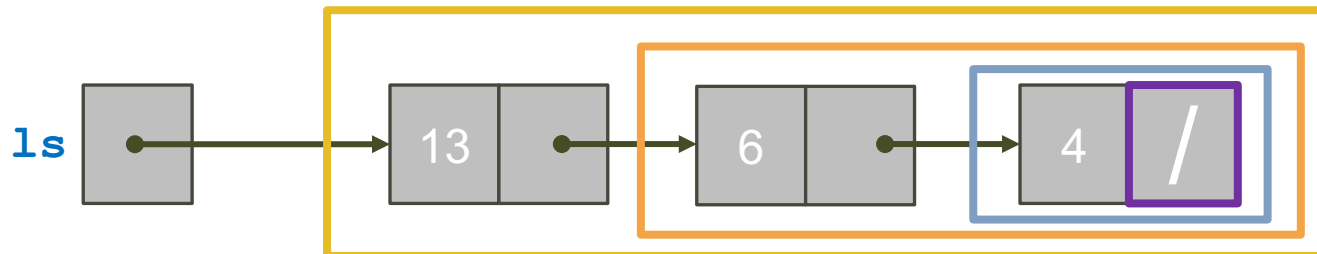


```
int sumIntList(List L) {  
    if (L == NULL)  
        return 0;  
  
    return L->val  
        + sumIntList(L->next);  
}
```



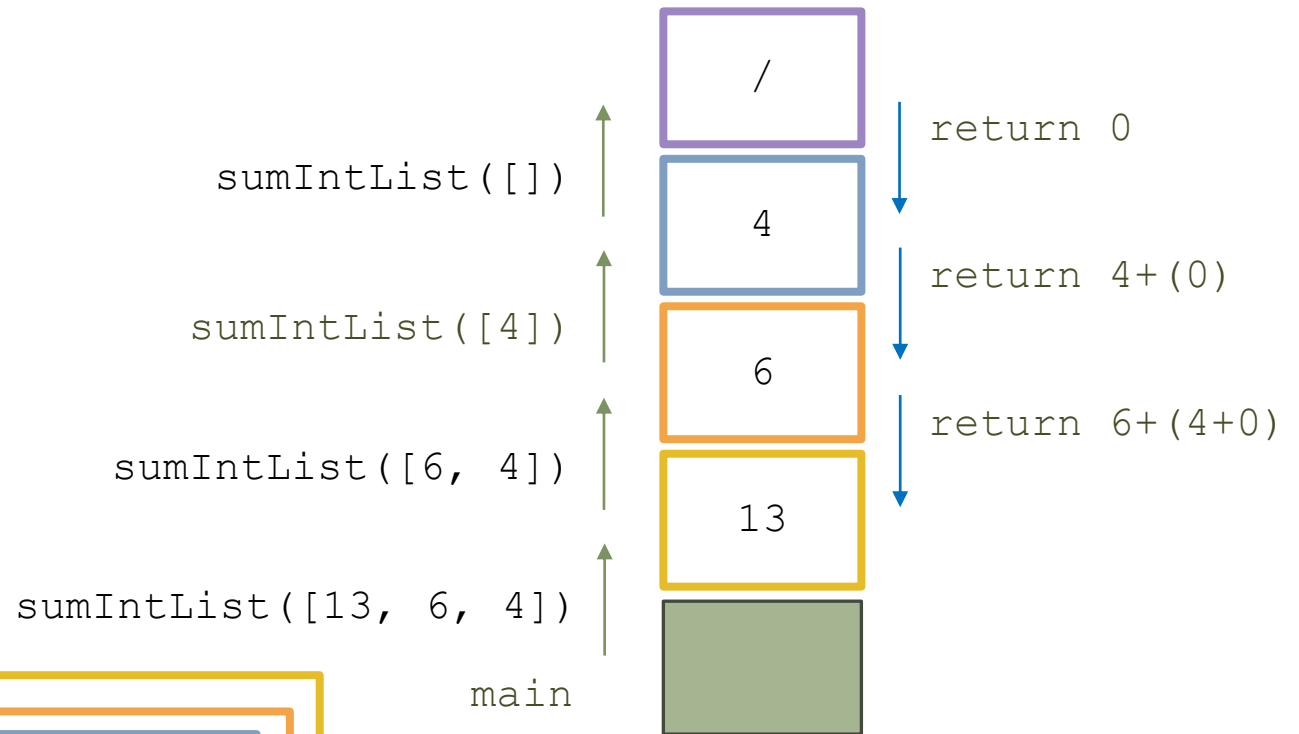
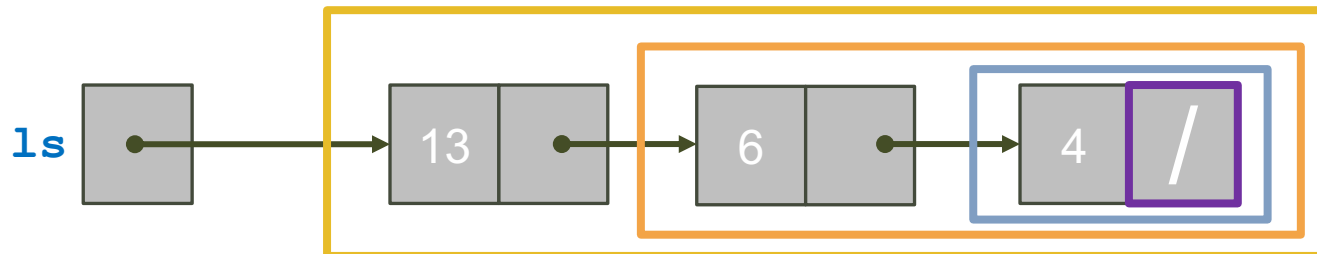
Simulazione: `sumIntList()`

```
int sumIntList(List L) {  
    if (L == NULL)  
        return 0;  
  
    return L->val  
        + sumIntList(L->next);  
}
```



Simulazione: `sumIntList()`

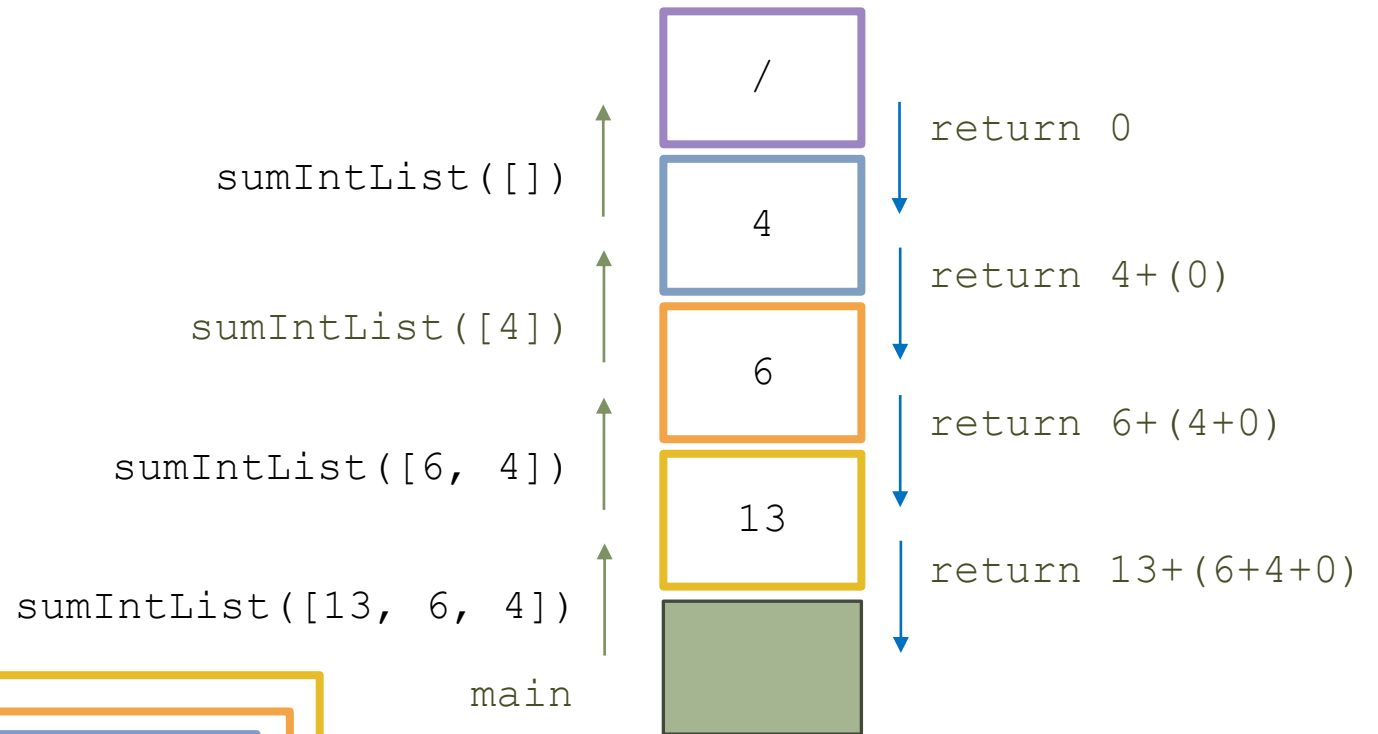
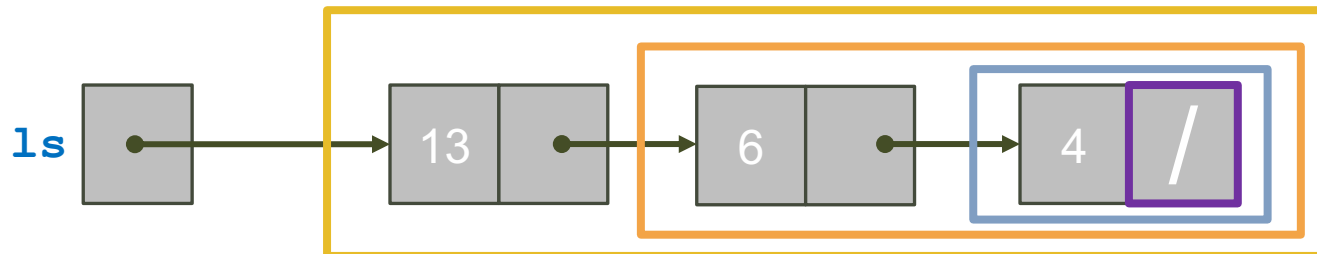
```
int sumIntList(List L) {  
    if (L == NULL)  
        return 0;  
  
    return L->val  
        + sumIntList(L->next);  
}
```



Simulazione: `sumIntList()`



```
int sumIntList(List L) {  
    if (L == NULL)  
        return 0;  
  
    return L->val  
        + sumIntList(L->next);  
}
```



Qual è lo svantaggio di `sumIntList()` ?



- Complessità computazionale?

-
-
-
-
-

```
int sumIntList(List L) {  
    if (L == NULL)  
        return 0;  
  
    return L->val  
        + sumIntList(L->next);  
}
```

Qual è lo svantaggio di `sumIntList()` ?



- Complessità computazionale?

- Tempo:

-

-

-

-

```
int sumIntList(List L) {  
    if (L == NULL)  
        return 0;  
  
    return L->val  
        + sumIntList(L->next);  
}
```

Qual è lo svantaggio di `sumIntList()` ?



- Complessità computazionale?

- Tempo:

- $O(N)$ con lista di N elementi

-

-

-

```
int sumIntList(List L) {  
    if (L == NULL)  
        return 0;  
  
    return L->val  
        + sumIntList(L->next);  
}
```

Qual è lo svantaggio di `sumIntList()` ?



- Complessità computazionale?

- Tempo:
 - $O(N)$ con lista di N elementi
- Spazio:
 -
 -

```
int sumIntList(List L) {  
    if (L == NULL)  
        return 0;  
  
    return L->val  
        + sumIntList(L->next);  
}
```

Qual è lo svantaggio di `sumIntList()` ?



- Complessità computazionale?

- Tempo:
 - $O(N)$ con lista di N elementi
- Spazio:
 - $O(N)$ con lista di N elementi

```
int sumIntList(List L) {  
    if (L == NULL)  
        return 0;  
  
    return L->val  
        + sumIntList(L->next);  
}
```


Qual è lo svantaggio di `sumIntList()` ?



- Complessità computazionale?
 - Tempo:
 - $O(N)$ con lista di N elementi
 - Spazio:
 - $O(N)$ con lista di N elementi
 - Non è una funzione ricorsiva di coda

```
int sumIntList(List L) {  
    if (L == NULL)  
        return 0;  
  
    return L->val  
        + sumIntList(L->next);  
}
```

Qual è lo svantaggio di `sumIntList()` ?

- Complessità computazionale?
 - Tempo:
 - $O(N)$ con lista di N elementi
 - Spazio:
 - $O(N)$ con lista di N elementi
 - Non è una funzione ricorsiva di coda

```
int sumIntList(List L) {  
    if (L == NULL)  
        return 0;  
  
    return L->val  
        + sumIntList(L->next);  
}
```

Ricorsione non di coda

sumIntList() Versione 2

- Rendiamo la funzione ricorsiva di coda
 - Complessità:
 - Tempo: $O(N)$
 - Uguale a prima
 - Spazio: $O(1)$
 - Permette ottimizzazione automatica da parte del compilatore
 -
 -
 -

sumIntList() Versione 2

- Rendiamo la funzione ricorsiva di coda
 - Complessità:
 - Tempo: $O(N)$
 - Uguale a prima
 - Spazio: $O(1)$
 - Permette ottimizzazione automatica da parte del compilatore
- Come?
 -
 -

sumIntList() Versione 2

- Rendiamo la funzione ricorsiva di coda
 - Complessità:
 - Tempo: $O(N)$
 - Uguale a prima
 - Spazio: $O(1)$
 - Permette ottimizzazione automatica da parte del compilatore
- Come?
 - Aggiungo accumulatore
 -

sumIntList() Versione 2



Accumulatore

- Rendiamo la funzione ricorsiva di coda
 - Complessità:
 - Tempo: $O(N)$
 - Uguale a prima
 - Spazio: $O(1)$
 - Permette ottimizzazione automatica da parte del compilatore
- Come?
 - Aggiungo accumulatore
 -

```
int sumIntListV2(List L, int acc) {  
    if (L == NULL)  
        return s;  
  
    return sumIntListV2(L->next,  
                        acc + L->val);  
}
```

sumIntList() Versione 2



Accumulatore

- Rendiamo la funzione ricorsiva di coda
 - Complessità:
 - Tempo: $O(N)$
 - Uguale a prima
 - Spazio: $O(1)$
 - Permette ottimizzazione automatica da parte del compilatore
 - Come?
 - Aggiungo accumulatore
 - Funzione wrapper

```
int sumIntListV2(List L, int acc) {  
    if (L == NULL)  
        return s;  
  
    return sumIntListV2(L->next,  
                        acc + L->val);  
}
```

sumIntList() Versione 2



- Rendiamo la funzione ricorsiva di coda

- Complessità:

- Tempo: $O(N)$
 - Uguale a prima
- Spazio: $O(1)$
 - Permette ottimizzazione automatica da parte del compilatore

- Come?

- Aggiungo accumulatore
- Funzione wrapper

Accumulatore

```
int sumIntListV2(List L, int acc) {  
    if (L == NULL)  
        return s;  
  
    return sumIntListV2(L->next,  
                        acc + L->val);  
}  
  
int somma(List L) {  
    return sumIntListV2(L, 0);  
}
```

Funzione wrapper

sumIntList() Versione 2

- Rendiamo la funzione ricorsiva di coda

- Complessità:

- Tempo: $O(N)$
 - Uguale a prima
- Spazio: $O(1)$
 - Permette ottimizzazione automatica da parte del compilatore

- Come?

- Aggiungo accumulatore
- Funzione wrapper

Accumulatore

```
int sumIntListV2(List L, int acc) {  
    if (L == NULL)  
        return s;  
  
    return sumIntListV2(L->next,  
                        acc + L->val);  
}  
  
int somma(List L) {  
    return sumIntListV2(L, 0);  
}
```

Ricorsione di coda

Funzione wrapper

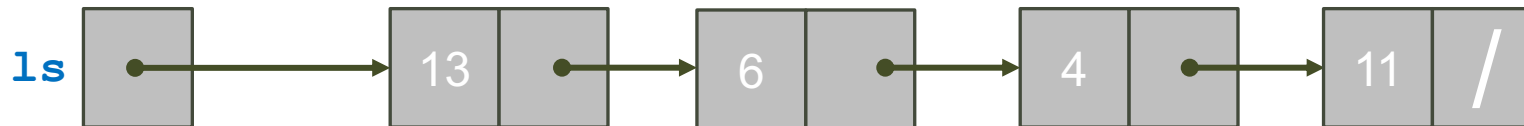
Esercizio: Operazioni di Visita

- Implementate in modo ricorsivo anche:
 - `void printIntList(IntList ls);`
 - Stampa gli elementi della lista
 - `size_t length(List ls);`
 - Restituisce il numero degli elementi della lista
 - `size_t count(List ls, Bool (*predicate)(T e));`
 - Restituisce il numero degli elementi della lista che soddisfa il predicato 'predicate'

Rimozione Lista Linkata

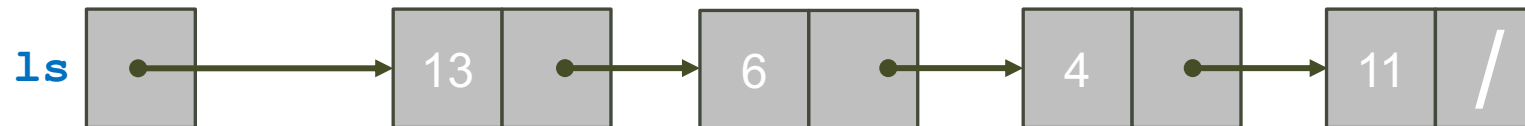


- Rimuovere una lista linkata significa:
 - Non solo togliere tutti gli elementi da essa
 - Ma liberare anche la memoria da essi occupata
- Varie strategie possibili:
 - Rimozione in testa
 - Facile da fare in modo iterativo
 - Esercizio per voi
 - Rimozione in coda



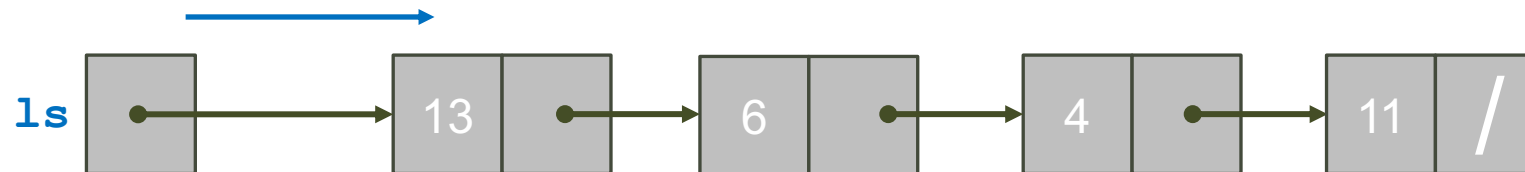
Lista Linkata: Rimozione in coda I

- Rimozione in coda:
 - Richiede di liberare gli elementi dell'esempio da destra a sinistra
 - Come?
- Idea:
 - Funzione ricorsiva con chiamate a free sugli elementi a ritroso



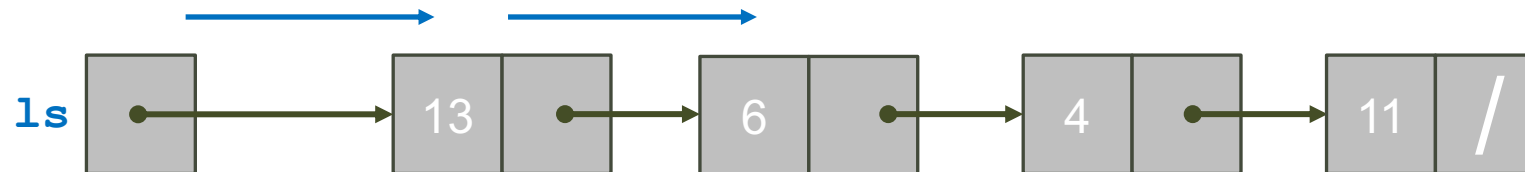
Lista Linkata: Rimozione in coda I

- Rimozione in coda:
 - Richiede di liberare gli elementi dell'esempio da destra a sinistra
 - Come?
- Idea:
 - Funzione ricorsiva con chiamate a free sugli elementi a ritroso



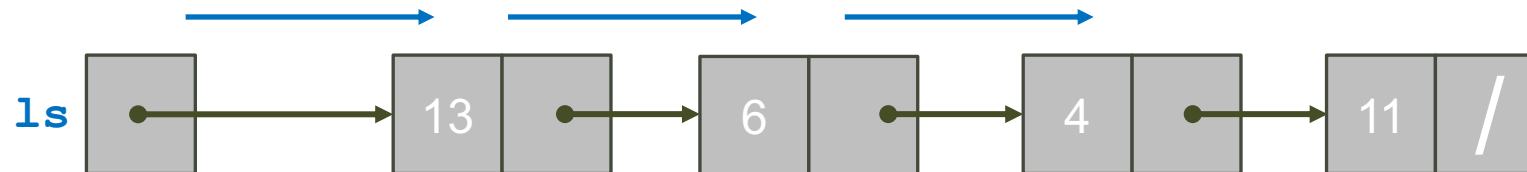
Lista Linkata: Rimozione in coda I

- Rimozione in coda:
 - Richiede di liberare gli elementi dell'esempio da destra a sinistra
 - Come?
- Idea:
 - Funzione ricorsiva con chiamate a free sugli elementi a ritroso



Lista Linkata: Rimozione in coda I

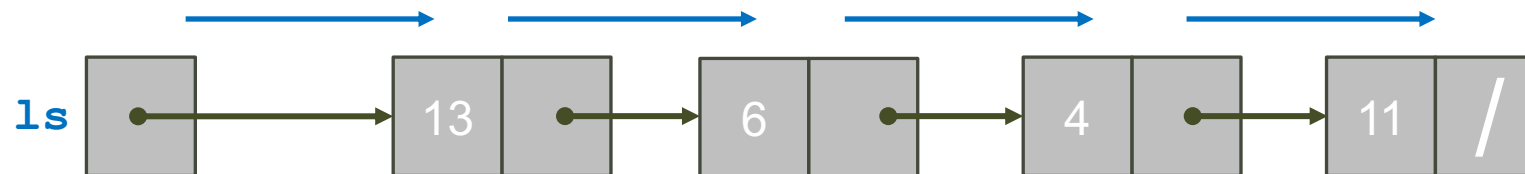
- Rimozione in coda:
 - Richiede di liberare gli elementi dell'esempio da destra a sinistra
 - Come?
- Idea:
 - Funzione ricorsiva con chiamate a free sugli elementi a ritroso



Lista Linkata: Rimozione in coda I

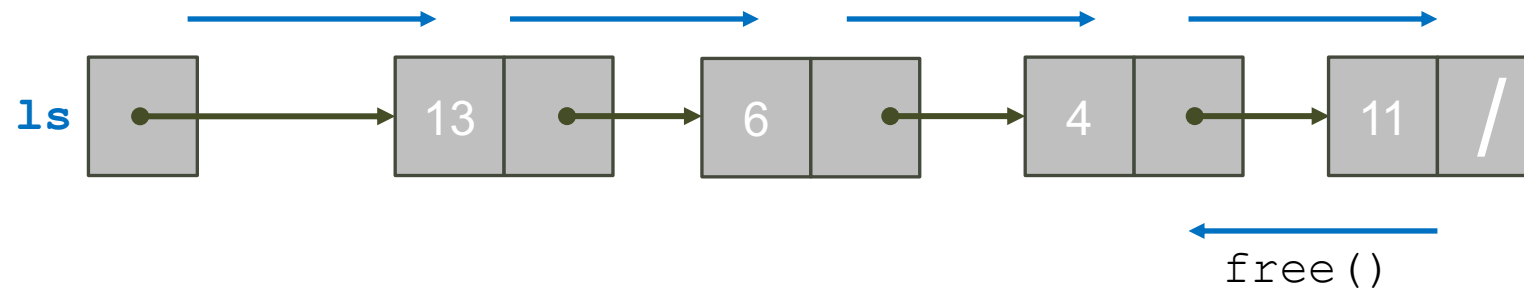


- Rimozione in coda:
 - Richiede di liberare gli elementi dell'esempio da destra a sinistra
 - Come?
- Idea:
 - Funzione ricorsiva con chiamate a free sugli elementi a ritroso



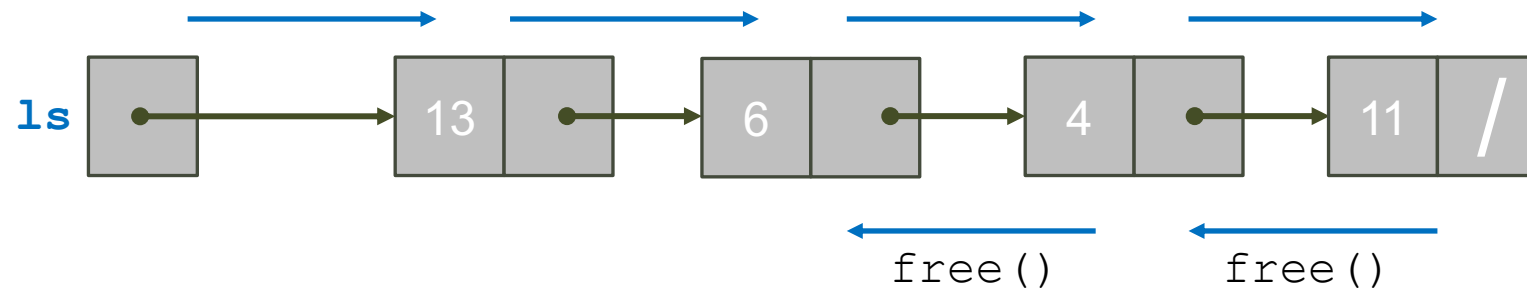
Lista Linkata: Rimozione in coda I

- Rimozione in coda:
 - Richiede di liberare gli elementi dell'esempio da destra a sinistra
 - Come?
- Idea:
 - Funzione ricorsiva con chiamate a free sugli elementi a ritroso



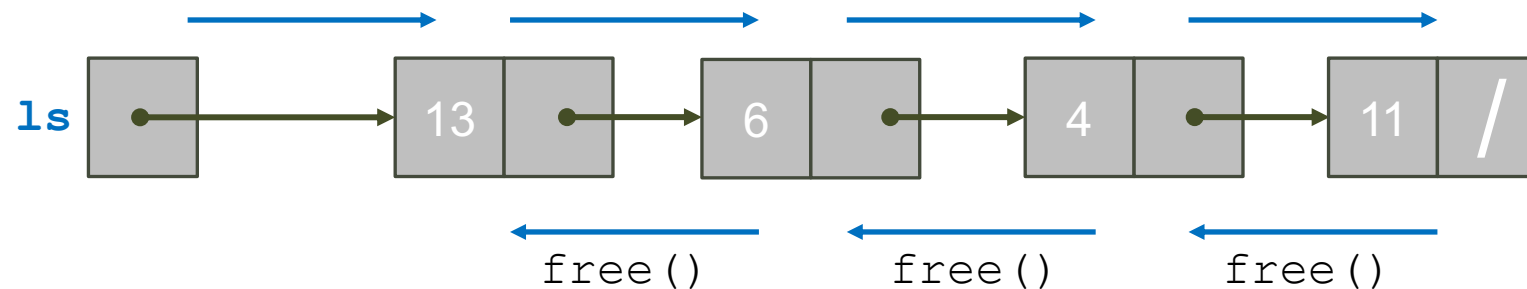
Lista Linkata: Rimozione in coda I

- Rimozione in coda:
 - Richiede di liberare gli elementi dell'esempio da destra a sinistra
 - Come?
- Idea:
 - Funzione ricorsiva con chiamate a free sugli elementi a ritroso



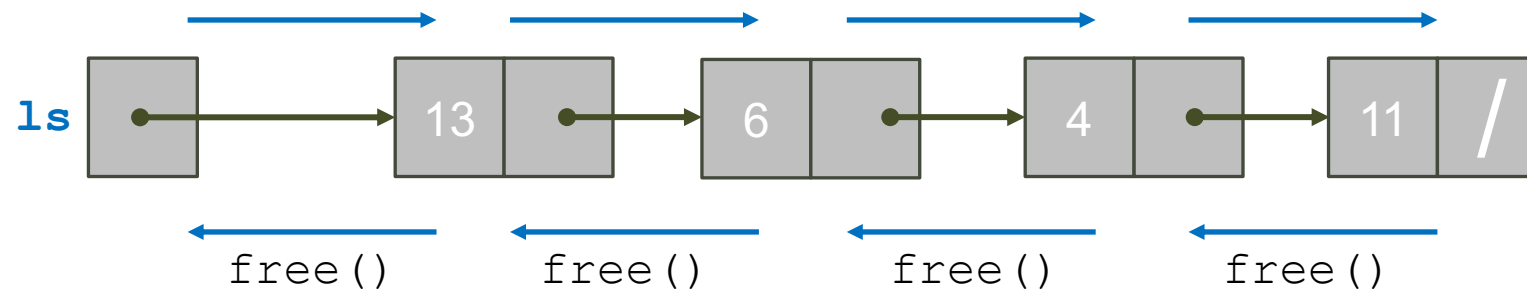
Lista Linkata: Rimozione in coda I

- Rimozione in coda:
 - Richiede di liberare gli elementi dell'esempio da destra a sinistra
 - Come?
- Idea:
 - Funzione ricorsiva con chiamate a free sugli elementi a ritroso



Lista Linkata: Rimozione in coda I

- Rimozione in coda:
 - Richiede di liberare gli elementi dell'esempio da destra a sinistra
 - Come?
- Idea:
 - Funzione ricorsiva con chiamate a free sugli elementi a ritroso



Lista Linkata: Rimozione in coda II

- Idea:
 - Funzione ricorsiva con chiamate a free sugli elementi a ritroso

```
List deleteList(List L) {  
    // se la lista non è vuota  
    if (L != NULL) {  
        // vai al prossimo elemento  
        L->next = deleteList(L->next);  
        // al ritorno libera il nodo  
        free(L);  
    }  
    return NULL; // segno di aver raggiunto la  
                coda: dopo non viene più nulla  
}
```

Lista Linkata: Rimozione in coda II

- Idea:
 - Funzione ricorsiva con chiamate a free sugli elementi a ritroso

Possiamo fare meglio?

```
List deleteList(List L) {  
    // se la lista non è vuota  
    if (L != NULL) {  
        // vai al prossimo elemento  
        L->next = deleteList(L->next);  
        // al ritorno libera il nodo  
        free(L);  
    }  
    return NULL; // segno di aver raggiunto la  
                coda: dopo non viene più nulla  
}
```

Lista Linkata: Rimozione in coda V2

- Miglioria:
 - Passiamo lista per riferimento
 -
 -
 -
 -

```
void deleteListV2(List *Lptr) {  
    // se il parametro è definito  
    if (Lptr != NULL) {  
        // se la lista non è vuota  
        if (*Lptr != NULL) {  
            // mi sposto avanti  
            deleteListV2(&((*Lptr)->next));  
            // libero il nodo (attuale ultimo)  
            free(*Lptr);  
            // segno che dopo non viene più nulla  
            (*Lptr) = NULL;  
        }  
    }  
}
```

Lista Linkata: Rimozione in coda V2

- Miglioria:
 - Passiamo lista per riferimento
 -
 -
 -
 -

```
void deleteListV2(List *Lptr) {  
    // se il parametro è definito  
    if (Lptr != NULL) {  
        // se la lista non è vuota  
        if (*Lptr != NULL) {  
            // mi sposto avanti  
            deleteListV2(&((*Lptr)->next));  
            // libero il nodo (attuale ultimo)  
            free(*Lptr);  
            // segno che dopo non viene più nulla  
            (*Lptr) = NULL;  
        }  
    }  
}
```

Cosa passo?

Lista Linkata: Rimozione in coda V2

- Miglioria:
 - Passiamo lista per riferimento
 -
 -
 -
 -

```
void deleteListV2(List *Lptr) {  
    // se il parametro è definito  
    if (Lptr != NULL) {  
        // se la lista non è vuota  
        if (*Lptr != NULL) {  
            // mi sposto avanti  
            deleteListV2(&((*Lptr)->next));  
            // libero il nodo (attuale ultimo)  
            free(*Lptr);  
            // segno che dopo non viene più nulla  
            (*Lptr) = NULL;  
        }  
    }  
}
```

Cosa passo?

- Memoria che contiene next

Lista Linkata: Rimozione in coda V2

- Miglioria:
 - Passiamo lista per riferimento
 - Vantaggi:
 -
 -
 -

```
void deleteListV2(List *Lptr) {  
    // se il parametro è definito  
    if (Lptr != NULL) {  
        // se la lista non è vuota  
        if (*Lptr != NULL) {  
            // mi sposto avanti  
            deleteListV2(&((*Lptr)->next));  
            // libero il nodo (attuale ultimo)  
            free(*Lptr);  
            // segno che dopo non viene più nulla  
            (*Lptr) = NULL;  
        }  
    }  
}
```

Cosa passo?

- Memoria che contiene next

Lista Linkata: Rimozione in coda V2

- Miglioria:
 - Passiamo lista per riferimento
 - Vantaggi:
 - Possiamo aggiornare la testa della lista

```
void deleteListV2(List *Lptr) {  
    // se il parametro è definito  
    if (Lptr != NULL) {  
        // se la lista non è vuota  
        if (*Lptr != NULL) {  
            // mi sposto avanti  
            deleteListV2(&((*Lptr)->next));  
            // libero il nodo (attuale ultimo)  
            free(*Lptr);  
            // segno che dopo non viene più nulla  
            (*Lptr) = NULL;  
        }  
    }  
}
```

Cosa passo?

- Memoria che contiene next

Lista Linkata: Rimozione in coda V2

- Miglioria:
 - Passiamo lista per riferimento
 - Vantaggi:
 - Possiamo aggiornare la testa della lista
 - Svantaggio che rimane?
 -

```
void deleteListV2(List *Lptr) {  
    // se il parametro è definito  
    if (Lptr != NULL) {  
        // se la lista non è vuota  
        if (*Lptr != NULL) {  
            // mi sposto avanti  
            deleteListV2(&((*Lptr)->next));  
            // libero il nodo (attuale ultimo)  
            free(*Lptr);  
            // segno che dopo non viene più nulla  
            (*Lptr) = NULL;  
        }  
    }  
}
```

Cosa passo?

- Memoria che contiene next

Lista Linkata: Rimozione in coda V2

- Miglioria:
 - Passiamo lista per riferimento
 - Vantaggi:
 - Possiamo aggiornare la testa della lista
 - Svantaggio che rimane?
 - Ricorsione non di coda

```
void deleteListV2(List *Lptr) {  
    // se il parametro è definito  
    if (Lptr != NULL) {  
        // se la lista non è vuota  
        if (*Lptr != NULL) {  
            // mi sposto avanti  
            deleteListV2(&((*Lptr)->next));  
            // libero il nodo (attuale ultimo)  
            free(*Lptr);  
            // segno che dopo non viene più nulla  
            (*Lptr) = NULL;  
        }  
    }  
}
```

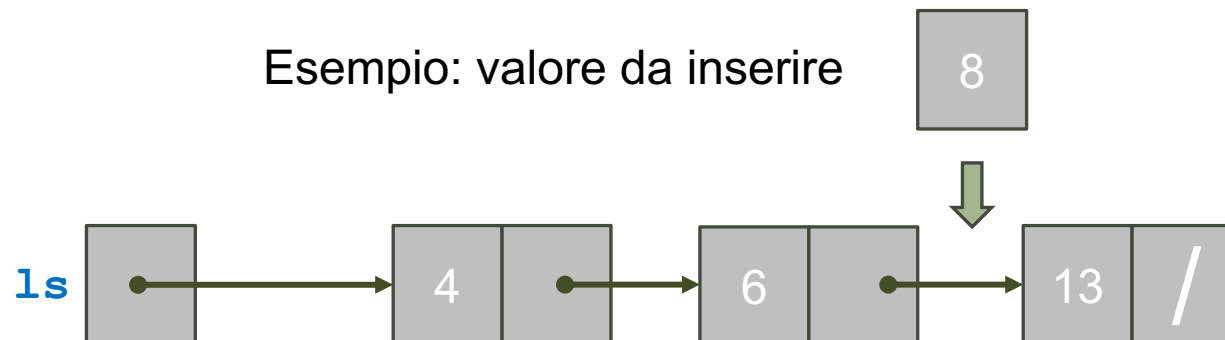
Cosa passo?

- Memoria che contiene next

Inserzione Nodo in Lista Ordinata I



- Consideriamo:
 - `_Bool insertInSortedIntList(IntList *ls, int value);`
 - Inserisce al posto giusto in una lista ordinata
 - Dopo l'inserzione la lista deve rimanere ordinata
 - Restituisce l'esito:
 - Se malloc restituisce NULL, allora non è possibile allocare nuova memoria e l'inserimento fallisce



Inserzione Nodo in Lista Ordinata II

- Casi base:
 - Parametro ls invalido (NULL)
 - Fallisce
 - La lista è vuota
 - Valore da inserire è minore del valore della lista
 - Inserzione in testa
-
-
-

```
_Bool insertInSortedIntList(IntList *ls, int v) {  
    if (NULL == ls) return 0; // parametro non valido  
    if ((NULL == *ls) || (v <= (*ls)->v) {  
        // Caso base: la lista è / o <hd, tl> con v <= hd->v:  
  
    }  
  
}
```

Inserzione Nodo in Lista Ordinata II

- Casi base:
 - Parametro ls invalido (NULL)
 - Fallisce
 - La lista è vuota
 - Valore da inserire è minore del valore della lista
 - Inserzione in testa
-
-
-

```
_Bool insertInSortedIntList(IntList *ls, int v) {  
    if (NULL == ls) return 0; // parametro non valido  
    if ((NULL == *ls) || (v <= (*ls)->v) {  
        // Caso base: la lista è / o <hd, tl> con v <= hd->v:  
        List new = malloc(sizeof(IntListNode));  
        if (new == NULL)  
            return 0; // Non c'è spazio disponibile  
    }  
  
}
```

Inserzione Nodo in Lista Ordinata II

- Casi base:
 - Parametro ls invalido (NULL)
 - Fallisce
 - La lista è vuota
 - Valore da inserire è minore del valore della lista
 - Inserzione in testa
-
-
-

```
_Bool insertInSortedIntList(IntList *ls, int v) {  
    if (NULL == ls) return 0; // parametro non valido  
    if ((NULL == *ls) || (v <= (*ls)->v) {  
        // Caso base: la lista è / o <hd, tl> con v <= hd->v:  
        List new = malloc(sizeof(IntListNode));  
        if (new == NULL)  
            return 0; // Non c'è spazio disponibile  
        // Inserimento in testa  
        new->v = v;  
        new->next = *ls;  
        *ls = new;  
        return 1;  
    }  
  
}
```

Inserzione Nodo in Lista Ordinata II

- Casi base:
 - Parametro ls invalido (NULL)
 - Fallisce
 - La lista è vuota
 - Valore da inserire è minore del valore della lista
 - Inserzione in testa
- Caso induttivo:
 -
 -

```
_Bool insertInSortedIntList(IntList *ls, int v) {  
    if (NULL == ls) return 0; // parametro non valido  
    if ((NULL == *ls) || (v <= (*ls)->v) {  
        // Caso base: la lista è / o <hd, tl> con v <= hd->v:  
        List new = malloc(sizeof(IntListNode));  
        if (new == NULL)  
            return 0; // Non c'è spazio disponibile  
        // Inserimento in testa  
        new->v = v;  
        new->next = *ls;  
        *ls = new;  
        return 1;  
    }  
  
}
```

Inserzione Nodo in Lista Ordinata II

- Casi base:
 - Parametro ls invalido (NULL)
 - Fallisce
 - La lista è vuota
 - Valore da inserire è minore del valore della lista
 - Inserzione in testa
- Caso induttivo:
 - Il valore da inserire è maggiore del valore della lista
 -

```
_Bool insertInSortedIntList(IntList *ls, int v) {  
    if (NULL == ls) return 0; // parametro non valido  
    if ((NULL == *ls) || (v <= (*ls)->v) {  
        // Caso base: la lista è / o <hd, tl> con v <= hd->v:  
        List new = malloc(sizeof(IntListNode));  
        if (new == NULL)  
            return 0; // Non c'è spazio disponibile  
        // Inserimento in testa  
        new->v = v;  
        new->next = *ls;  
        *ls = new;  
        return 1;  
    }  
  
}
```


Inserzione Nodo in Lista Ordinata II

- Casi base:
 - Parametro ls invalido (NULL)
 - Fallisce
 - La lista è vuota
 - Valore da inserire è minore del valore della lista
 - Inserzione in testa
- Caso induttivo:
 - Il valore da inserire è maggiore del valore della lista
 - Inserzione nel resto della lista

```
_Bool insertInSortedIntList(IntList *ls, int v) {  
    if (NULL == ls) return 0; // parametro non valido  
    if ((NULL == *ls) || (v <= (*ls)->v) {  
        // Caso base: la lista è / o <hd, tl> con v <= hd->v:  
        List new = malloc(sizeof(IntListNode));  
        if (new == NULL)  
            return 0; // Non c'è spazio disponibile  
        // Inserimento in testa  
        new->v = v;  
        new->next = *ls;  
        *ls = new;  
        return 1;  
    }  
}
```

Inserzione Nodo in Lista Ordinata II

- Casi base:
 - Parametro ls invalido (NULL)
 - Fallisce
 - La lista è vuota
 - Valore da inserire è minore del valore della lista
 - Inserzione in testa
- Caso induttivo:
 - Il valore da inserire è maggiore del valore della lista
 - Inserzione nel resto della lista

```
_Bool insertInSortedIntList(IntList *ls, int v) {  
    if (NULL == ls) return 0; // parametro non valido  
    if ((NULL == *ls) || (v <= (*ls)->v) {  
        // Caso base: la lista è / o <hd, tl> con v <= hd->v:  
        List new = malloc(sizeof(IntListNode));  
        if (new == NULL)  
            return 0; // Non c'è spazio disponibile  
        // Inserimento in testa  
        new->v = v;  
        new->next = *ls;  
        *ls = new;  
        return 1;  
    }  
    // Caso induttivo: la lista è <hd, tl> con v > hd->v.  
    // Inserzione in tl.  
    return insertInSortedIntList(&((*ls)->next), v);  
}
```

Simulazione: insertInSortedIntList()



```
_Bool insertInSortedIntList(IntList *ls,
int v) {
    if (NULL == ls)
        return 0; // parametro non valido
    if ((NULL == *ls) || (v <= (*ls)->v) {
        // Caso base: inserzione in testa
        [...]
        return 1;
    }
    // Caso induttivo: inserzione in tl.
    return insertInSortedIntList(&((*ls)-
>next), v);
}
```

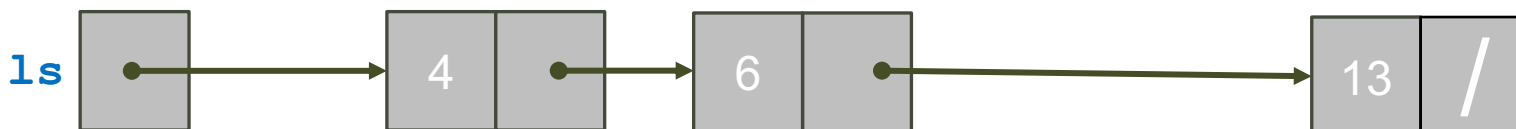


Simulazione: insertInSortedIntList()



```
_Bool insertInSortedIntList(IntList *ls,
int v) {
    if (NULL == ls)
        return 0; // parametro non valido
    if ((NULL == *ls) || (v <= (*ls)->v) {
        // Caso base: inserzione in testa
        [...]
        return 1;
    }
    // Caso induttivo: inserzione in tl.
    return insertInSortedIntList(&((*ls)-
>next), v);
}
```

main



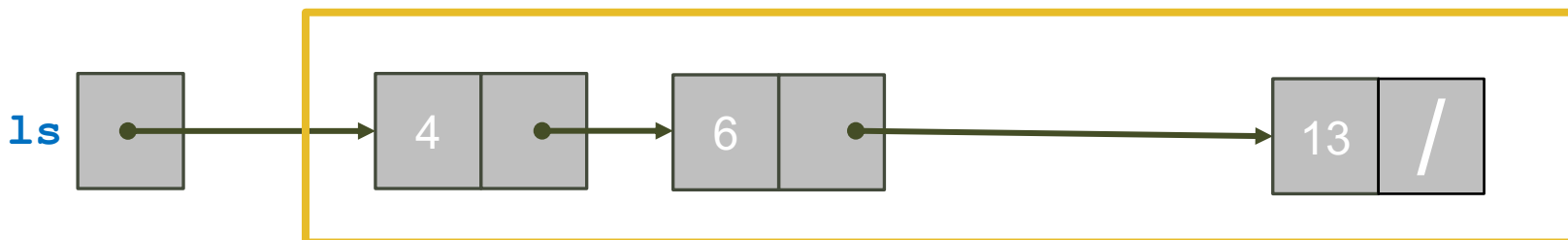
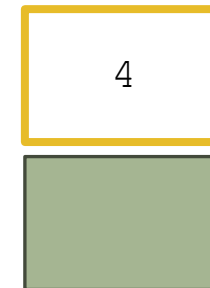
Simulazione: insertInSortedIntList()



```
_Bool insertInSortedIntList(IntList *ls,
int v) {
    if (NULL == ls)
        return 0; // parametro non valido
    if ((NULL == *ls) || (v <= (*ls)->v) {
        // Caso base: inserzione in testa
        [...]
        return 1;
    }
    // Caso induttivo: inserzione in tl.
    return insertInSortedIntList(&((*ls)-
>next), v);
}
```

insertInSortedIntList([4,6,13], 8)

main



Simulazione: insertInSortedIntList()

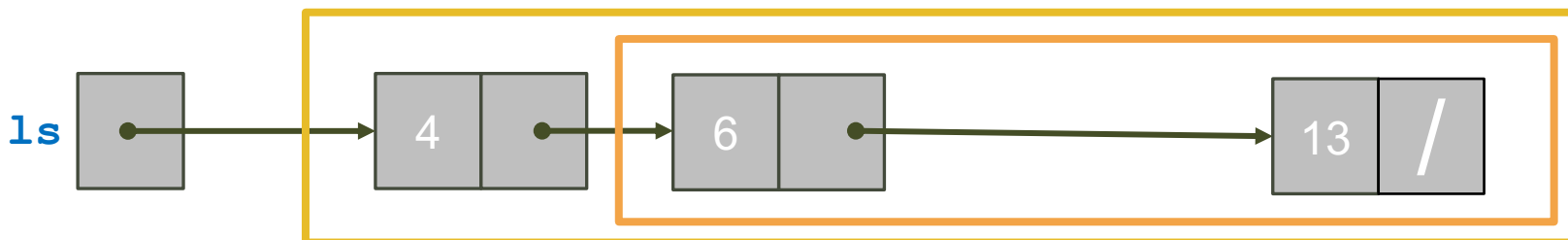


```
_Bool insertInSortedIntList(IntList *ls,
int v) {
    if (NULL == ls)
        return 0; // parametro non valido
    if ((NULL == *ls) || (v <= (*ls)->v) {
        // Caso base: inserzione in testa
        [...]
        return 1;
    }
    // Caso induttivo: inserzione in tl.
    return insertInSortedIntList(&((*ls)-
>next), v);
}
```

insertInSortedIntList([6,13], 8)

insertInSortedIntList([4,6,13], 8)

main



Simulazione: `insertInSortedIntList()`

```
_Bool insertInSortedIntList(IntList *ls,
int v) {
    if (NULL == ls)
        return 0; // parametro non valido
    if ((NULL == *ls) || (v <= (*ls)->v) {
        // Caso base: inserzione in testa
        [...]
        return 1;
    }
    // Caso induttivo: inserzione in tl.
    return insertInSortedIntList(&(*ls)-
>next), v);
}
```

`insertInSortedIntList([13], 8)`

`insertInSortedIntList([6,13], 8)`

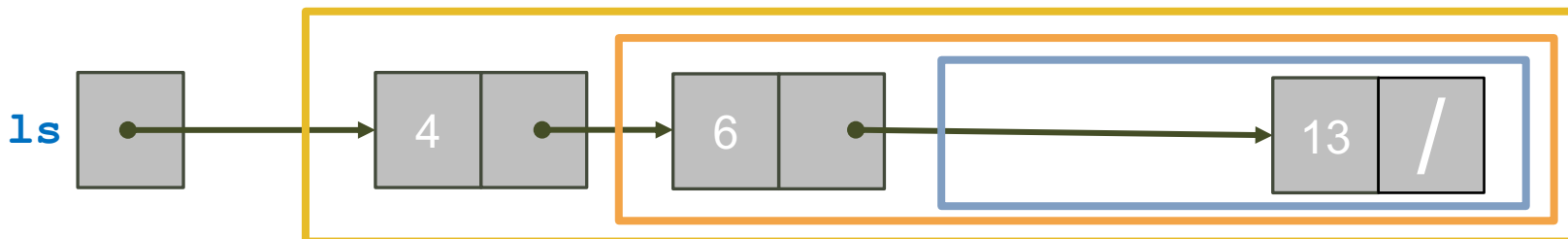
`insertInSortedIntList([4,6,13], 8)`

main

13

6

4



Simulazione: insertInSortedIntList()



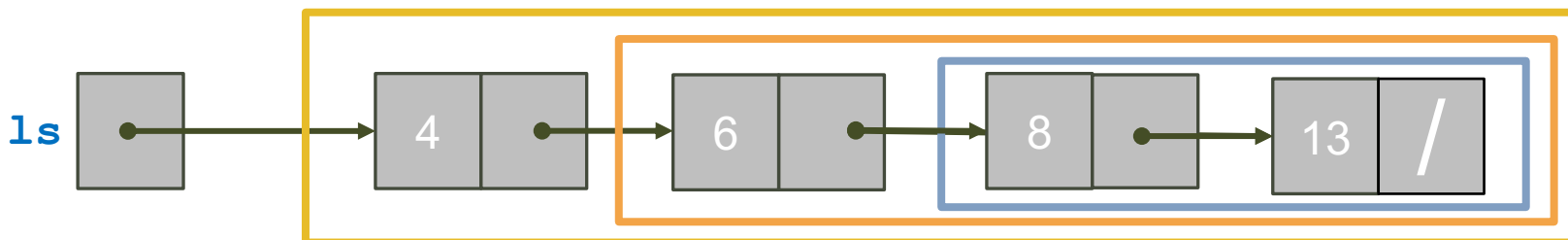
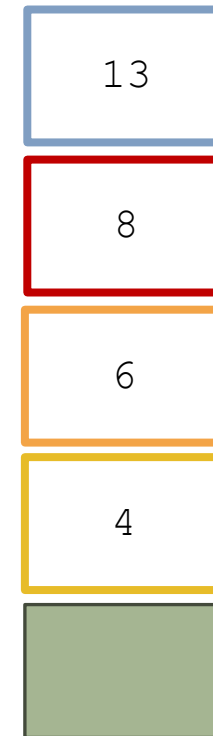
```
_Bool insertInSortedIntList(IntList *ls,
int v) {
    if (NULL == ls)
        return 0; // parametro non valido
    if ((NULL == *ls) || (v <= (*ls)->v) {
        // Caso base: inserzione in testa
        [...]
        return 1;
    }
    // Caso induttivo: inserzione in tl.
    return insertInSortedIntList(&(*ls)-
>next), v);
}
```

insertInSortedIntList([13], 8)

insertInSortedIntList([6,13], 8)

insertInSortedIntList([4,6,13], 8)

main



Simulazione: insertInSortedIntList()

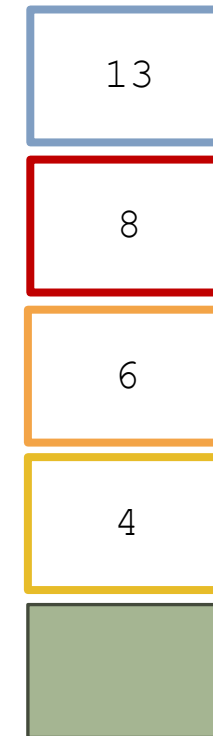
```
_Bool insertInSortedIntList(IntList *ls,
int v) {
    if (NULL == ls)
        return 0; // parametro non valido
    if ((NULL == *ls) || (v <= (*ls)->v) {
        // Caso base: inserzione in testa
        [...]
        return 1;
    }
    // Caso induttivo: inserzione in tl.
    return insertInSortedIntList(&((*ls)-
>next), v);
}
```

insertInSortedIntList([13], 8)

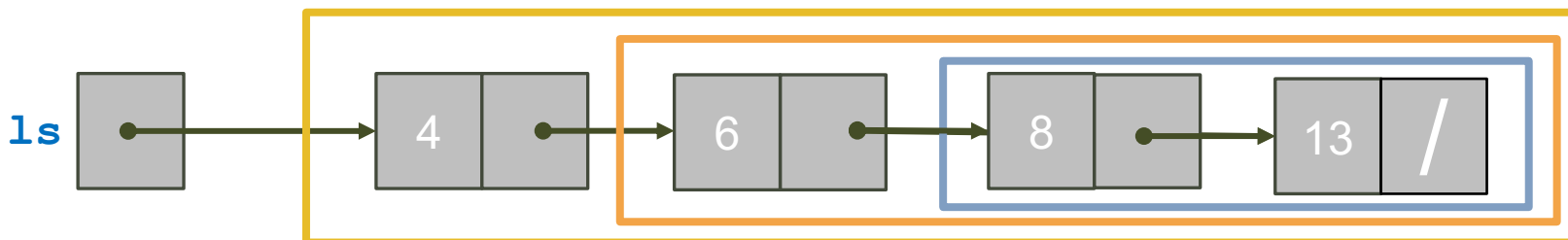
insertInSortedIntList([6,13], 8)

insertInSortedIntList([4,6,13], 8)

main



return 1



Simulazione: `insertInSortedIntList()`

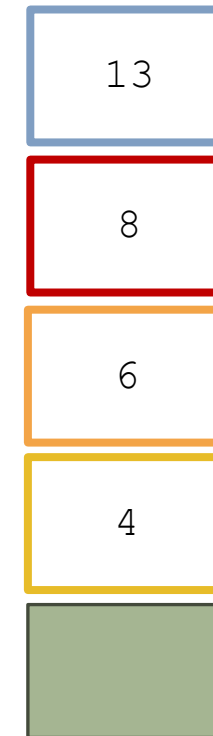
```
_Bool insertInSortedIntList(IntList *ls,
int v) {
    if (NULL == ls)
        return 0; // parametro non valido
    if ((NULL == *ls) || (v <= (*ls)->v) {
        // Caso base: inserzione in testa
        [...]
        return 1;
    }
    // Caso induttivo: inserzione in tl.
    return insertInSortedIntList(&(*ls)-
>next), v);
}
```

`insertInSortedIntList([13], 8)`

`insertInSortedIntList([6,13], 8)`

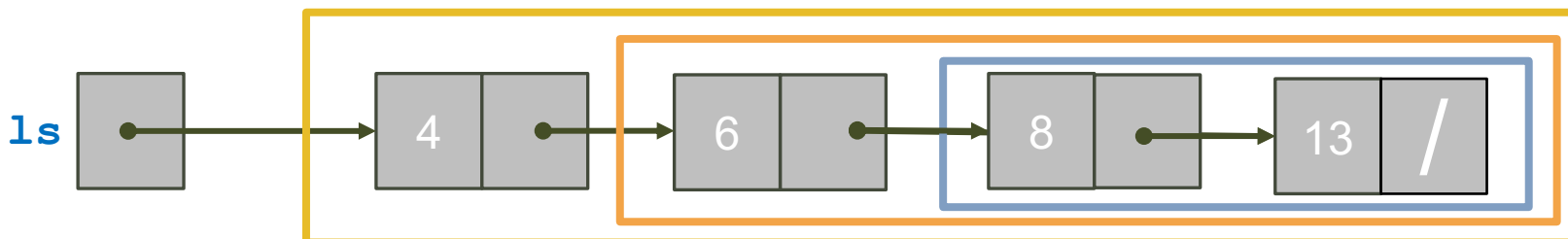
`insertInSortedIntList([4,6,13], 8)`

main



return 1

return 1



Simulazione: insertInSortedIntList()

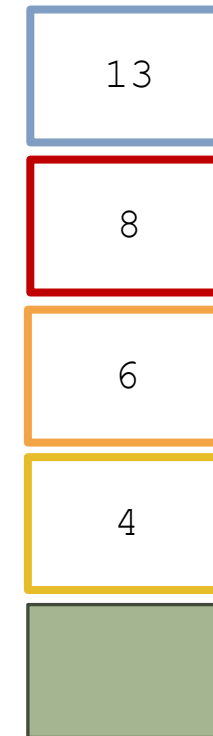
```
_Bool insertInSortedIntList(IntList *ls,
int v) {
    if (NULL == ls)
        return 0; // parametro non valido
    if ((NULL == *ls) || (v <= (*ls)->v) {
        // Caso base: inserzione in testa
        [...]
        return 1;
    }
    // Caso induttivo: inserzione in tl.
    return insertInSortedIntList(&(*ls)-
>next), v);
}
```

insertInSortedIntList([13], 8)

insertInSortedIntList([6,13], 8)

insertInSortedIntList([4,6,13], 8)

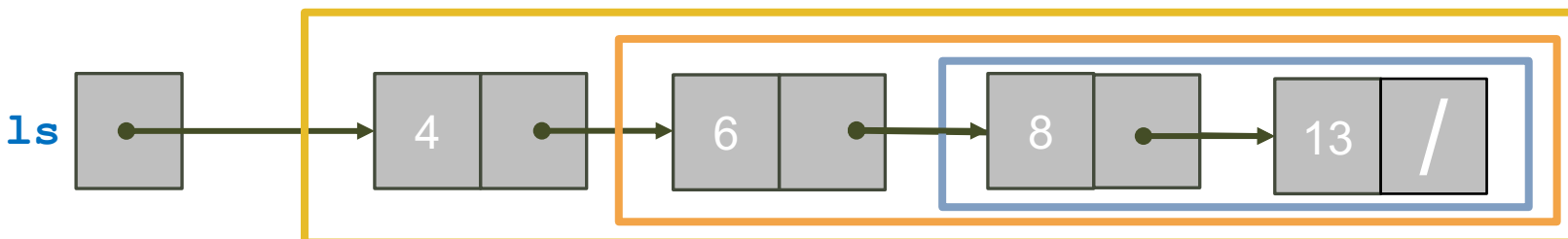
main



return 1

return 1

return 1

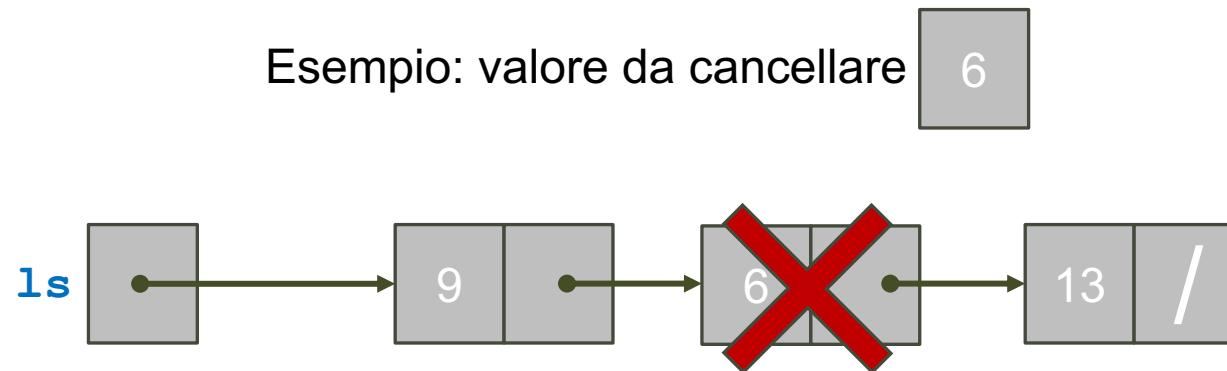


Esercizio: Operazioni di Inserimento

- Implementate in modo ricorsivo anche:
 - `_Bool insertBefore(List *lsPtr, T e, List positionPtr);`
 - Inserisce prima del nodo puntato da positionPtr (se positionPtr == NULL, inserisce in coda)
 - `_Bool insertAfter(List *lsPtr, T e, List positionPtr);`
 - Inserisce dopo il nodo puntato da positionPtr (se positionPtr == NULL, inserisce in testa)
- Le operazioni restituiscono l'esito
 - Se malloc restituisce NULL, allora non è possibile allocare nuova memoria e l'inserimento fallisce

Cancellazione di un Elemento

- Consideriamo:
 - `_Bool deleteFromIntList(IntList *ls, int value);`
 - Cancella dalla lista `*ls` la prima occorrenza dell'elemento `value`
 - Restituisce l'esito:
 - Se l'elemento da cancellare non è presente, allora la cancellazione fallisce



Cancellazione di un Elemento

- Caso base 1:

```
_Bool deleteFromIntList(IntList *ls, int v) {
```


Cancellazione di un Elemento

- Caso base 1:
 - La lista è vuota
 - Elemento non presente

```

_Bool deleteFromIntList(IntList *ls, int v) {
    // Caso base 1: la lista è /, v non è presente in lista
    if ((ls == NULL) || (*ls == NULL)) return 0;
}

```

Cancellazione di un Elemento

- Caso base 1:
 - La lista è vuota
 - Elemento non presente
- Caso base 2:
 - Testa contiene v
 -
 -
 -
 -
 -

```

_Bool deleteFromIntList(IntList *ls, int v) {
    // Caso base 1: la lista è /, v non è presente in lista
    if ((ls == NULL) || (*ls == NULL)) return 0;
}

```

Cancellazione di un Elemento

- Caso base 1:
 - La lista è vuota
 - Elemento non presente
- Caso base 2:
 - Testa contiene v
 -
 -
 -

```
_Bool deleteFromIntList(IntList *ls, int v) {  
    // Caso base 1: la lista è /, v non è presente in lista  
    if ((ls == NULL) || (*ls == NULL)) return 0;  
  
    // Caso base 2: la lista <hd, tl> con hd->v == v,  
  
  
  
  
  
  
  
  
  
}
```

Cancellazione di un Elemento

- Caso base 1:
 - La lista è vuota
 - Elemento non presente
- Caso base 2:
 - Testa contiene v
 - Cancellazione in testa
- -
 -
 -

```

_Bool deleteFromIntList(IntList *ls, int v) {
    // Caso base 1: la lista è /, v non è presente in lista
    if ((ls == NULL) || (*ls == NULL)) return 0;

    // Caso base 2: la lista ⟨hd, tl⟩ con hd->v == v,
}

```

Cancellazione di un Elemento

- Caso base 1:
 - La lista è vuota
 - Elemento non presente
- Caso base 2:
 - Testa contiene v
 - Cancellazione in testa
 -
 -
 -

```
_Bool deleteFromIntList(IntList *ls, int v) {  
    // Caso base 1: la lista è /, v non è presente in lista  
    if ((ls == NULL) || (*ls == NULL)) return 0;  
  
    // Caso base 2: la lista <hd, tl> con hd->v == v,  
    // cancellazione in testa  
    if ((*ls)->v == v) {  
        tmp = *ls;  
        *ls = (*ls)->next;  
        free(tmp);  
        return 1;  
    }  
}
```

Cancellazione di un Elemento

- Caso base 1:
 - La lista è vuota
 - Elemento non presente
- Caso base 2:
 - Testa contiene v
 - Cancellazione in testa
- Caso induttivo:
 - Testa non contiene v
 -

```
_Bool deleteFromIntList(IntList *ls, int v) {  
    // Caso base 1: la lista è /, v non è presente in lista  
    if ((ls == NULL) || (*ls == NULL)) return 0;  
  
    // Caso base 2: la lista <hd, tl> con hd->v == v,  
    // cancellazione in testa  
    if ((*ls)->v == v) {  
        tmp = *ls;  
        *ls = (*ls)->next;  
        free(tmp);  
        return 1;  
    }  
  
}
```

Cancellazione di un Elemento

- Caso base 1:
 - La lista è vuota
 - Elemento non presente
- Caso base 2:
 - Testa contiene v
 - Cancellazione in testa
- Caso induttivo:
 - Testa non contiene v
 -

```
_Bool deleteFromIntList(IntList *ls, int v) {  
    // Caso base 1: la lista è /, v non è presente in lista  
    if ((ls == NULL) || (*ls == NULL)) return 0;  
  
    // Caso base 2: la lista <hd, tl> con hd->v == v,  
    // cancellazione in testa  
    if ((*ls)->v == v) {  
        tmp = *ls;  
        *ls = (*ls)->next;  
        free(tmp);  
        return 1;  
    }  
    // Caso induttivo: la lista è <hd, tl> con hd->v /= v  
  
}
```

Cancellazione di un Elemento

- Caso base 1:
 - La lista è vuota
 - Elemento non presente
- Caso base 2:
 - Testa contiene v
 - Cancellazione in testa
- Caso induttivo:
 - Testa non contiene v
 - Cancellazione nel resto della lista

```
_Bool deleteFromIntList(IntList *ls, int v) {  
    // Caso base 1: la lista è /, v non è presente in lista  
    if ((ls == NULL) || (*ls == NULL)) return 0;  
  
    // Caso base 2: la lista <hd, tl> con hd->v == v,  
    // cancellazione in testa  
    if ((*ls)->v == v) {  
        tmp = *ls;  
        *ls = (*ls)->next;  
        free(tmp);  
        return 1;  
    }  
    // Caso induttivo: la lista è <hd, tl> con hd->v != v  
}
```


Cancellazione di un Elemento

- Caso base 1:
 - La lista è vuota
 - Elemento non presente
- Caso base 2:
 - Testa contiene v
 - Cancellazione in testa
- Caso induttivo:
 - Testa non contiene v
 - Cancellazione nel resto della lista

```
_Bool deleteFromIntList(IntList *ls, int v) {  
    // Caso base 1: la lista è /, v non è presente in lista  
    if ((ls == NULL) || (*ls == NULL)) return 0;  
  
    // Caso base 2: la lista <hd, tl> con hd->v == v,  
    // cancellazione in testa  
    if ((*ls)->v == v) {  
        tmp = *ls;  
        *ls = (*ls)->next;  
        free(tmp);  
        return 1;  
    }  
    // Caso induttivo: la lista è <hd, tl> con hd->v != v  
    // cancellazione di value da tl  
    return deleteFromIntList(&((*ls)->next), v);  
}
```

Esercizio: Operazioni di Inserimento

- Implementate in modo ricorsivo anche:
 - `_Bool delete(List *ls, List positionPtr);`
 - Cancella dalla lista `*ls` il nodo puntato da `positionPtr`
 - `_Bool deleteAfter(List *ls, List positionPtr);`
 - Cancella dalla lista `*ls` il nodo dopo il nodo puntato da `positionPtr`
 - Se `positionPtr == NULL`, cancella il nodo in testa
- Le operazioni restituiscono l'esito
 - Se l'elemento da cancellare non è presente, allora la cancellazione fallisce

Errori Tipici da Evitare

- Lista passata per valore
 - Dimenticare di verificare se `L` vale `NULL`
- Lista passata per riferimento
 - Dimenticare di fare la doppia verifica `if (Lptr) if (*Lptr)`
- Saltare un caso
- Non eseguire operazioni nel giusto ordine
- Dimenticare di assegnare `NULL` al next dell'ultimo elemento in coda
- Assegnare `NULL` al next di un elemento da inserire fra due nodi